University of Manchester
School of Computer Science
Project Report 2022

Alzheimer's Classification using Medical Images and Convolutional Neural Networks

Author: Eirik Fladmark

Supervisor: Dr. Fumie Costen

Abstract

Alzheimer's Classification using Medical Images and Convolutional Neural Networks

Author: Eirik Fladmark

This project devises an Machine Learning (ML) model for classifying Alzheimer's Disease (AD) from Magnetic Resonance Imaging (MRI) brain scans. It focuses on classifying the brain into one of three categories: AD, Mild Cognitive Impairment (MCI), and Normal Cognition (NC). AD is a neurodegenerative disease that slowly destroys memory and other cognitive abilities. AD is heavily researched and the search for a treatment that can stop its progression would be groundbreaking. The MCI stage is the step between NC and AD. This class will be of focus throughout the report, as there exists medication and treatments that can slow the progression from MCI to AD if discovered early enough. The report covers common MRI preprocessing methods and state-of-the-art classification techniques. The result of the project was a weighted ensemble ML model, that used three separate Convolutional Neural Networks (CNNs) to classify AD based on 2D slices extracted from the coronal, sagittal, and transverse plane of a 3D MRI scan. The architecture of the individual models was optimised through extensive hyperparameter tuning. The final model achieved an overall accuracy of 82.7%, with a 76.9% accuracy within the MCI class.

Supervisor: Dr. Fumie Costen

Acknowledgements

I would like to express my gratitude to everyone who has been involved in the process of completing this project. Furthermore, a special thanks to Fumie Costen, who has motivated and supported me from the very start.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Introduction to Subject

Alzheimer's Disease (AD) is a neurodegenerative disease that destroys the brain's neurotransmitter systems [17]. The disease is the most common cause of dementia; a disease that is estimated to affect over 45 million people worldwide [18]. The progression of the disease is classified into three main stages: Normal Cognition (NC), Mild Cognitive Impairment (MCI), and AD. MCI is the most difficult stage to classify, as it can easily be confused with normal aged forgetfulness [19]. However, as opposed to AD, evidence shows that drugs and cognitive training can slow the progression of MCI to AD. This makes the detection of MCI crucial for improving the lives of the affected individuals.

However, because of the aforementioned difficulty in classifying the MCI stage, computational methods such as Machine Learning (ML) have become popular. A commonly used ML network for AD prediction is the Convolutional Neural Network (CNN) [12][20][21][2][22]. The CNN is popular in medical applications because of its ability to effectively learn from medical imagery [23]. These networks are commonly trained on Magnetic Resonance Imaging (MRI) data, however, Computed Tomography (CT) and Positron Emission Tomography (PET) scans are also seen in recent research [24][20].

1.2 Project Description

This project will create an ML classifier to successfully classify the state of a brain into AD, MCI or NC based on the MRI brain scans. The result of the project is an optimised ensemble model based on the 2.5D CNN methodology presented by Dai [14], and the model architecture devised by Lackey [15].

1.3 Motivation

Alzheimer's Disease (AD) affects around 10% of adults of age 65 and above. Every five years after the age of 65 the risk of developing the disease doubles [25]. Mild Cognitive

Impairment (MCI) is an early stage of memory loss or other cognitive ability loss in individuals who still possess the ability to independently perform most activities of daily living [26]. Being able to detect the MCI stage opens up the possibility of treatment before the likely brain deterioration which leads to AD. Hence, being able to catch MCI early could severely lead to a better quality of life for both the affected individual, as well as for their family. Furthermore, detecting MCI using only Magnetic Resonance Imaging (MRI) scans and the human eye is very hard. However, by applying data preprocessing steps and Machine Learning (ML) methods it is possible to classify the state of a brain with high accuracy.

1.4 Aims and Objectives

1.4.1 Aims

The aim of the project is to formulate and create a model which is successfully able to classify the state of a brain into AD, MCI, or NC. This includes data preprocessing steps and effective ML methods to improve the model's performance. The report also aims to give a fundamental understanding of AD, and ML techniques used for the final model.

1.4.2 Objectives

1. Data Preprocessing
   (a) Understand preprocessing techniques used to transform the MRI data into appropriate model input for the chosen architecture.
   (b) Criteria for success: Use the previously preprocessed dataset from Lackey [15] and understand the steps taken to acquire it.
2. CNN
   (a) Create or improve an existing CNN architecture such that it can accurately classify a brain into AD, MCI, or NC.
   (b) Criteria for success: The accuracy of the model should exceed that of previous years, this means improving the 80.7% accuracy acquired by Lackey [15].
3. MCI Performance
   (a) The conversion from MCI to AD can be slowed down through different treatments and medication. Hence, being able to discover MCI in a patient can increase the quality of life of the patient significantly. Therefore, this report has an increased focus on the performance of the MCI class.
   (b) Criteria for success: The MCI class accuracy should exceed that of previous years, this means improving the 72.4% accuracy acquired by Lackey [15]
4. Small Dataset

(a) A challenge within medical applications is often the limited size of the available datasets. This is also the case for the data acquired for this project, hence it is important to try to achieve good results despite this limitation.

(b) Criteria for success: Get reasonable results using the acquired dataset

1.4.3 Report Structure

The report has been structured with the intent to clearly explain all concepts concisely before they are referenced within the work.

Chapter 2: Background

The background section is divided into two sections:

- Alzheimer's Disease (AD)

  This section will introduce AD and explain what is known about the disease, from causes and stages to diagnosis and treatments. The section will also look more in-depth at MCI, as this is the most crucial stage to diagnose due to the existing treatments.

- Machine Learning (ML)

  In this section a brief introduction to ML is given, together with concepts which are relevant to this project. The chapter describes the ML pipeline and covers important topics surrounding data and ML tasks. Important base ML knowledge will be established, and important concepts relating to the project are outlined, some of the most important concepts include Convolutional Neural Networks (CNNs) and hyperparameters.

Chapter 3: Literary Review

This chapter will cover previous work done in the field of AD prediction using ML. It will first go into state-of-the-art preprocessing methods and what they achieve. It will then move on to describe the most used ML networks and models used for classifying AD using MRI scans, and other brain biomarkers. A summary of the result achieved by the different studies is provided. Lastly, the chapter covers the preprocessing steps used to achieve the dataset used for this project, as provided through work by Lackey [15] and Dai [14].

Chapter 4: Research Methods

This chapter shortly outlines the methods and tools used for the project. It shows the initial model architecture and addresses how experiments are run. Lastly, is mentions some considerations around hyperparameter tuning.

Chapter 5: Model Construction

This chapter introduces the concept of ensemble methods and how they have been used on the model. Following this, the chapter goes into detail on each of the hyperparameters, evaluating their performance to decide which are the best-performing ones. A good trade-off between time-complexity (often correlated to the memory requirement) and performance is what is sought after. The initial hyperparameters are introduced at the start of the chapter, whilst the final and best-performing hyperparameters are shown after all the experiments.

Chapter 6: Results and Discussion

This chapter analyses and discuss the results obtained during the model construction phase. It presents confusion matrices and model performance metrics and compares these to the performance of previous years. Lastly, it evaluates the set objectives and reflects on the project as a whole.

Chapter 7: Conclusion and Future Work

This chapter outlines the most important aspects of the project. Following this, it discusses steps that can help improve the current state of the model, as well as other approaches which could be worth investigating for the task of AD classification.

Chapter 2

Background

2.1 Alzheimer's Disease

Alzheimer's Disease (AD) is a disease characterised by several degenerative changes in the brain's neurotransmitter systems [17]. These include a progressive decline in two or more cognitive domains including, but not limited to, memory, language, visuospatial function, and loss of abilities to perform instrumental and/or basic daily living activities [18]. AD is the most common cause of dementia worldwide and the prevalence of dementia is estimated to be over 45 million people worldwide. Furthermore, due to the world's increasing life expectancy, it is predicted to triple by 2050 [18] [27]. AD accounts for around 60% to 80% of all cases of dementia [28].

2.1.1 Causes

Scientists do not yet fully understand what it is that explicitly causes AD in most people [29]. It is likely to include a combination of metrics, such as changes in the brain due to age, along with environmental, gentic, and lifestyle factors [29]. However, AD's neurodegenerative attributes are well known. It is characterised by neurotic plaques and neurofibrillary tangles (seen in Figure 2.1) which come as a result of Amyloid-Beta Peptide (Aβ) accumulation in the medial temporal lobe and the neocortical structures [1]. Recent studies have indicated that this accumulation of Aβ also accelerates AD progression by further increasing degeneration and loss/death of neurons [30]. However, whether this accumulation is a cause or a consequence of AD is still not certain [30]. Progressive loss of cognitive functions can be caused by cerebral disorders like AD, however, factors such as intoxications, infections, reduced oxygen supply to the brain, nutritional deficiency etc. can also play a role [1].

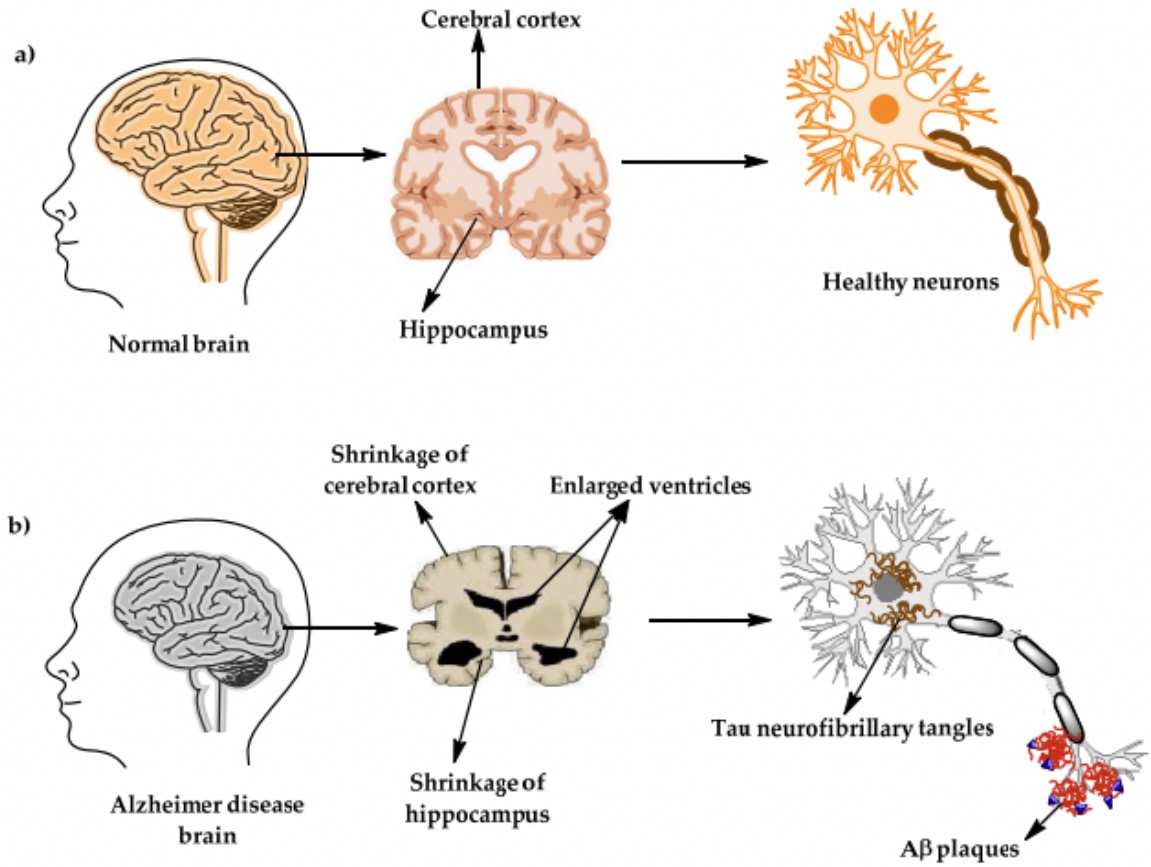


Figure 2.1: The physiological structure of the brain and neurons in (a) Healthy brain and (b) AD affected brain [1]

2.1.2 Stages

AD tends to develop slowly, and whilst the disease worsens over time, the rate at which it progresses varies between individuals [31]. Reisberg and Franssen [19] split the progression of AD into 7 stages. Stages 1 and 2 include normal cognitive ability and normal cognitive ability with aged forgetfulness. The latter mainly describes an individual over the age of 65 who feels that they remember worse than what they did 5 to 10 years. In this report, these two stages are generalised under the class NC. The following stage of the progression to AD, is MCI. How someone falls into this category is described further in subsubsection 2.1.2.1. The last 4 stages go all the way from mild AD to severe AD. These stages range from forgetting a recent visit to a friend or relative, all the way to being incapable of sitting upright without assistance - these stages are categorised under AD.

2.1.2.1 Criteria for MCI

When trying to categorise the stages of AD it is essential to understand the criteria that define a certain class. MCI is the bridge between Normal Cognition (NC) and AD and it is, therefore, crucial to properly define it. This is supported by studies showing that based

on the classification criteria, the difference in brains identified as MCI versus NC brains ranged all the way from 11% to 74% [32]. The data used for this project originates from the Alzheimer's Disease Neuroimaging Initiative (ADNI). ADNI uses the same criteria for MCI as defined in [33]. According to the ADNI protocol, a diagnosis of MCI is given if a participant, clinician or, study partner reported cognitive issues, if the patient showed some form of impairment on the logical memory-II subtest from the Wechsler memory scale-R, an Mini-Mental State Examination (MMSE) score equal to or above 24, a clinical dementia rating equal to 0.5, and general cognitive and functional performance sufficiently preserved such that a diagnosis of dementia could not be made at the time of the screening [33][34][32].

2.1.3 Diagnosis

The main problem with diagnosing AD lies within the definition of the disease itself [35]. The definition, as well as the manifestations of the disease, can often vary. The diagnosis of AD is clinicopathological, meaning that the diagnosis is based on a combination of both the symptoms as expressed by the patient, as well as the pathology of the disease. However, despite the challenges AD introduces, there are still a number of methods and tools used by professionals to try and diagnose AD [36]. The patient, as well as their close friends and family, are often questioned on the overall health and status of the patient. Furthermore, psychiatric tests are used to determine if other mental health conditions exist and if they could be a contributor to the patient's symptoms. This could also include other general medical tests (e.g. blood samples) to see if there are other underlying causes of the problem. Clinical ability assessments that focus on memory, problem-solving, and language can also be part of the evaluation. Brain scans such as CT, MRI, or PET are also often used to diagnose AD and can make it easier for the specialist to determine the current state of the patient's brain. This is useful not only for a potential diagnosis, but also to discard other possible sources of the symptoms.

2.1.4 Treatment

There is no cure for AD, however, there are steps that can be taken to manage the symptoms of the disease [37]. Management is supportive, with a focus on preserving dignity and providing care for as long as possible [38]. A list of the core treatments for AD as described in Kumar and Clark's book on Clinical Medicine can be found in Appendix B [38].

2.2 Machine Learning (ML)

Machine Learning (ML) is the study of computer algorithms that can improve automatically through experience, statistical methodologies, and (usually) large amounts of data [39]. These algorithms are approximations of the real world, as the core areas where ML is applied has an underlying algorithm which is often unknown. However, what is lacking in knowledge of the underlying algorithm, can be made up for in data [40]. There are certain patterns in the data, and whilst the algorithm's approximation might not explain

everything, these patterns are still useful when it comes to predicting the future; assuming that the close future, is not too different from the past [40]. In this project, ML is used to try to classify the state of a brain. There is no known algorithm for correctly determining whether a person has NC, MCI or AD. Hence, using labelled data, a model can be trained to approximate the theoretical algorithm which solves the problem.

2.2.1 Machine Learning Pipeline

The ML pipeline outlines the main steps which need to be taken to go from nothing to a functional ML model. Here are some questions that need to be answered to successfully accomplish this:

- Data gathering: Which data is relevant for the model's task? Where can such data be obtained? Is the source of our data reliable?
- Data pre-processing: How do we extract the best features from our obtained data? What format does our data need to be in for our model?
- Data usage: How do we choose to split our data? Should we use an equal amount of samples from each class?
- Model: What is an appropriate model given our goal? Which model architecture is proven good given our input data?
- Hyperparameter tuning: Which hyperparameters have the most impact on the model's result? Do we perform random search hyperparameter tuning, or nested hyperparameter tuning?
- Evaluation: How do we rank the performance of the model? Which type of cross-validation do we use (k-fold cross-validation, Leave-One-Out-Cross-Validation (LOOCV) etc.)? Do we get desired results?

The points outlined above are some of the many important questions that need to be answered during the development of an ML model.

2.2.2 Classification

The problem of classifying an MRI brain scan into classes based on their medical status is what is known as a classification problem. The task of classification falls under the ML domain called supervised learning. Supervised learning refers to the area of ML where the agent observes multiple input-output pairs and tries to learn the underlying function that maps the input to the output [41]. In the case of this model, the goal is to find the underlying function mapping a brain scan to either NC, MCI or AD.

2.2.3 Data

Together with sophisticated mathematical algorithms, data is one of the core components of ML applications. The more data one has, the better the ML model can learn to approximate reality. Keep in mind that any obtained data needs to meet very high-quality standards [42]. If a model is trained on bad data, the ML tools obtained from it are useless, and can in many ways be dangerous if assumed to have correct predictive authority [42]. More important concepts surrounding data are explained below.

2.2.3.1 Untrustworthy Data

As previously mentioned, the trustworthiness of data is essential. Data cleaning and preparation take about 80% of the average data scientist's time [43]. Even with all this time put into it, it is often hard to know whether all the errors in the data have been found [43]. A recent study found that 47% of newly created data had at least one critical error. The study also concluded that using the vaguest common standard, only 3% of the data quality evaluation methods could be rated as "acceptable" [43].

2.2.3.2 Data Size Requirements

For any predictive modelling problem, there is no defined amount of data needed. An increased amount of data improves models' performance, and help decrease bias and variance error [44]. However, the increased amount of data also further elevates training complexity [44]. There is no set size of data for a problem, however, the current argument seems to be on how many thousands of samples per class are needed for an acceptable model [45]. Hence, a reasonable assumption is that the number of samples per class should at least exceed a thousand.

2.2.3.3 The Curse of Dimensionality

The Curse of Dimensionality is a concept that refers to many of the different phenomenons that arise when analysing high-dimensional data. In pattern recognition problems, increasing the dimensionality of the data will reduce error up to a certain point, however, beyond a certain dimensionality, the performance starts to decrease, rather than increase [46]. This is particularly important when it comes to working with MRI brain scans. These are Three-dimensional (3D) scans, where each voxel can have a value between 0 to 255. Thus, it is important to be aware that the increased training time introduced by more complex data will not necessarily always pay off, and that sometimes reducing complexity can be the key to a successful model.

2.2.3.4 Validation Sets

In ML, data is often split into 3 separate sets, the training set, the validation set, and the test set. The training set and the test set are used for what their names suggest - the former is used for training the model, whilst the latter is used to test it. However, how can the training parameters of the model be evaluated before it is tested? This is where the validation set

steps in. During the model evaluation stage, the model's hyperparameters can be tuned to improve validation metrics. This is done to optimise the model's performance, whilst not exposing the test set to the model. The result of this is that the test set can give us an unbiased evaluation of the final model when the hyperparameter tuning is complete.

2.2.3.5 Data Augmentation

Data augmentation is a method used to synthetically create samples to increase the size of a dataset [47]. The augmented data can be used for training a machine learning classifier, giving it more input than it would have had otherwise [47]. Previous research demonstrates that data augmentation can also act as a regulariser in preventing neural networks to overfit, whilst also improving the performance of imbalanced class problems [47]. Different types of data augmentation include rotating, re-scaling, zooming, cropping, adding noise, and other augmentations that somewhat alter the initial data [48]. An example of data augmentation can be seen in Figure 2.2.

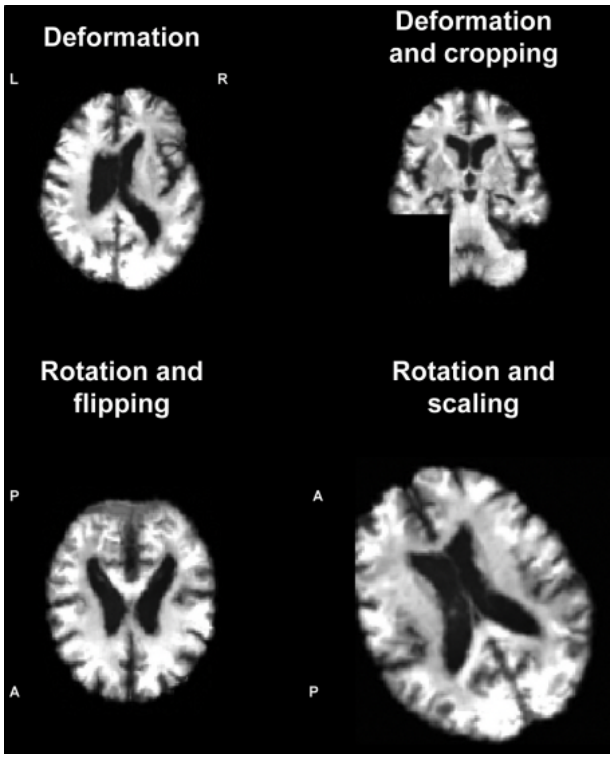


Figure 2.2: Examples of images after data augmentation. Axial and coronal images are shown. A = anterior; L = left; P = posterior; R = right. [2]

2.2.4 Model

For the AD classification problem, there are many different ML architectures and approaches that have been previously used. Some of the more basic approaches include the

Support Vector Machine (SVM), general logistic regression models, and decision tree algorithms [49]. More sophisticated approaches range all the way from Gradient Boosted Machines (GBMs) and CNNs, to more advanced Residual Neural Networks (ResNets) such as the ResNet-50 [50]. The input data of this project's model is image slices (extracted from MRIs scans), therefore, the implementation of a CNN is very appealing as it efficiently can capture the picture's complex and potentially non-linear patterns. Some essential aspects of the devised model are described in the following sections.

2.2.4.1 Artificial Neural Networks

An Artificial Neural Network (ANN) is a network of neurons (or nodes) inspired by the computational properties observed in biological organisms [51]. These networks consist of an input layer of neurons, followed by a hidden layer of neurons, and lastly a final layer of output neurons [52]. The concept of ANNs was first introduced in 1943 by Warren McCulloch and Walter Pitts in their famous paper, "A logical calculus of the ideas immanent in nervous activity" [53].

Each of the artificial neurons in the network receives one or multiple inputs and activates if more than a certain number of these inputs are active [54]. An improvement of these neurons, named perceptrons, was later introduced by Frank Rosenblatt in 1957 [54]. Instead of basing the input purely on binary on/off values, each of the inputs would now be numbers where each of these input connections had a weight [54]. The sum of these inputs is then passed through an activation function whose job is to transform the sum into the output of the neuron. Biological neurons were observed to roughly follow the sigmoid function, resulting in it being used for a long time. However, recent research indicates that the Rectified Linear Unit (ReLU) function generally performs better in ANNs [54]. After the introduction of backpropagation using gradient descent, these neurons could be successfully combined into bigger neural networks [54]. Backpropagation and gradient descent is explained more in-depth later.

A simple neural network can be seen in Figure 2.3, each of the circles represents a neuron, whilst the edges represent the weight a neuron has with respect to the neuron it is connected to. The neural network introduced later for predicting the classes NC, MCI, and AD uses pictures of dimensions 64×64 pixels, hence opposed to Figure 2.3, it has 64×64 input neurons, and 3 output neurons (one for each class) - the hidden layer is not strictly dependent on the input or output, however, this also differs as the used network is much deeper.

2.2.4.2 Training

For neural networks to learn from data, they need to go through a training process. This is where backpropagation with gradient descent steps in. For every training instance, the algorithm computes the output of every neuron in all layers consecutively (forward pass) [54]. It then calculates the error of the result, and how much each neuron in the previous hidden layer contributed to the error of this result - it continues to propagate backwards until it reaches the input layer (reverse pass) [54]. The last step is applying gradient descent, an

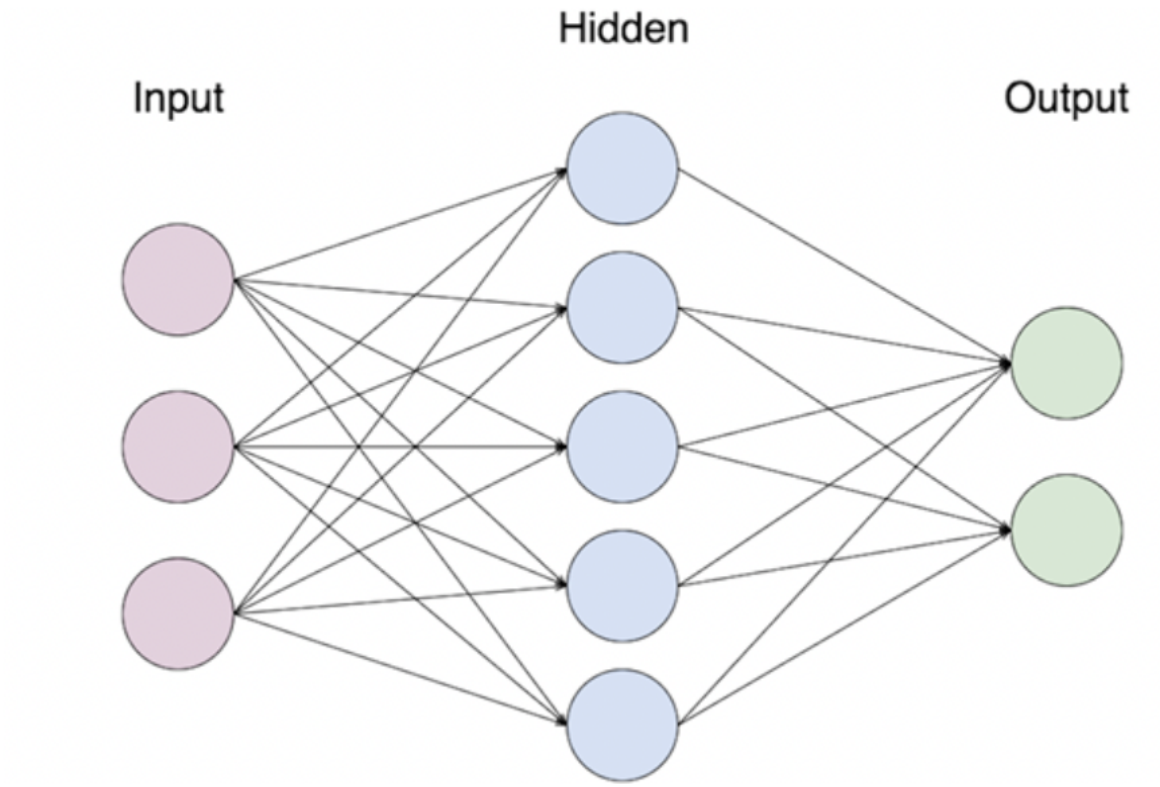


Figure 2.3: Schematic of a neural network with one hidden layer, from [3]

optimiser algorithm, that slightly tunes all the weights of the neurons against the gradient in an attempt to decrease the error [54].

2.2.4.3 Loss Function

The loss function (or error function) is a function that computes how far the predicted result (output) is from the expected result (the ground truth) [55]. The computed value is a distance score, measuring how well the network has done on the given example [55]. The fundamental utility of this score is to use it as a feedback signal during gradient descent to help tweak the weights slightly, in the direction that will lower the score for the given example [55].

2.2.4.4 Optimisers

During backpropagation an optimiser function is used to try and minimise the loss function of the neural network by tweaking weights towards the negative direction of their gradient [23]. The gradient descent optimiser has been mentioned previously, however, there exist a number of different and more effective implementations than the original algorithm [23]. One example of a more modern optimiser is the mini-batch Stochastic Gradient Decent (SGD) optimiser [23]. The original version of gradient descent would first run all training examples forward, then back through the network, before proceeding to update all the weights [23]. This can be inefficient for two main reasons:

(1) The weights of a network are often wrong to such a high degree (especially in the early stages of training) that only a small sample of training data can be used to create a good estimate of how much we want to tweak the weights.

(2) In networks with many hidden layers the number of parameters can quickly exceed the millions. This combined with big datasets gives the model a lot of information to go through before the network can update its weights [23].

Mini-batch SGD deals with this problem by picking a random batch of samples from the training data, training on these random samples, before updating the weights [23]. The normal SGD optimiser does the same, however, it only picks one sample at a time [23]. Mini-batch SGD is the middle ground between normal SGD and gradient descent. It is often considered to be a good option with a fair trade-off between stability, computational speed and memory requirements [23]. The Adam optimisation algorithm is another optimiser used in this project. Adam combines the best properties of the AdaGrad and RMSProp algorithms [56] (two extensions of the gradient descent algorithm) to better handle sparse gradients and noisy problems [57].

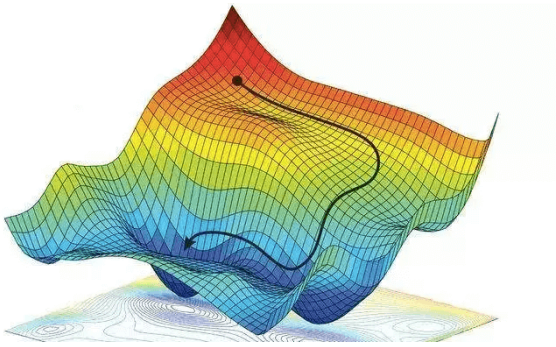


Figure 2.4: Visualisation of how a model moves towards the loss function's minima when tuning weights during gradient decent [4]

2.2.4.5 Deep Learning

The term Deep Learning (or Deep Neural Network) refers to ANNs with multiple hidden layers [7]. Recent technological advances and the network's ability to handle huge amounts of data have made it considered to be one of the decades' most powerful tools [7]. The extra hidden layers have proven to be particularly efficient at pattern recognition tasks [7]. One of the most popular deep neural networks is the CNN. This architecture is a key component in the model that is used in this project [7].

2.2.4.6 Convolutional Neural Networks

CNNs are networks created to work with grid-structured inputs where there is an important spatial dependency between the local regions of the grid [23]. Visual inputs such as pictures are one of the most obvious examples of what these networks are created for [23]. However, the network can also be effective on sequenced data such as text and time series that also show strong relationships with adjacent items. Some of the main methods used in CNNs

are described below using a 64 × 64 × 3 (the last value refers to the depth of the RGB channel) picture as reference:

- Convolution

  Convolution is an operation between a kernel (a grid structure with a set of weights) and a grid-structured input (often an image) [23]. Using the given image, a kernel would scan the picture from left-right and top-bottom, mapping every local region (limited by the size of the kernel) to a neuron in the next layer [7]. A fully mapped layer is often referred to as a feature map [23]. Doing this local generalisation of features can help reduce the number of weighted connections by a significant amount [7]. For example, if the layer following the input layer had 64 × 64 neurons and convolutions were not used, the result would be 64 × 64 × 3 by 64 × 64 = 50331648 weighted connections using the same approach as in [7]. However, assuming that the network is using a 5 × 5 × 3 kernel, each neuron in the output layer would only be dependent on 5 × 5 × 3 neurons in the original image. This would reduce the number of connections to 5 × 5 × 3 by 64 × 64 totalling only 307200 weighted connections. Since the kernel slides over all the local regions of the picture it is able to detect features in the image which are independent of where in the picture they are found - this is why the process is called convolution. It is also possible to add more layers after the input layer (increase depth). In doing so different features can be extracted from the same image as it can apply a different filter to each of the layers. A visualisation of how different convolutional filters can create different feature maps from the same image can be seen in Figure 2.8.

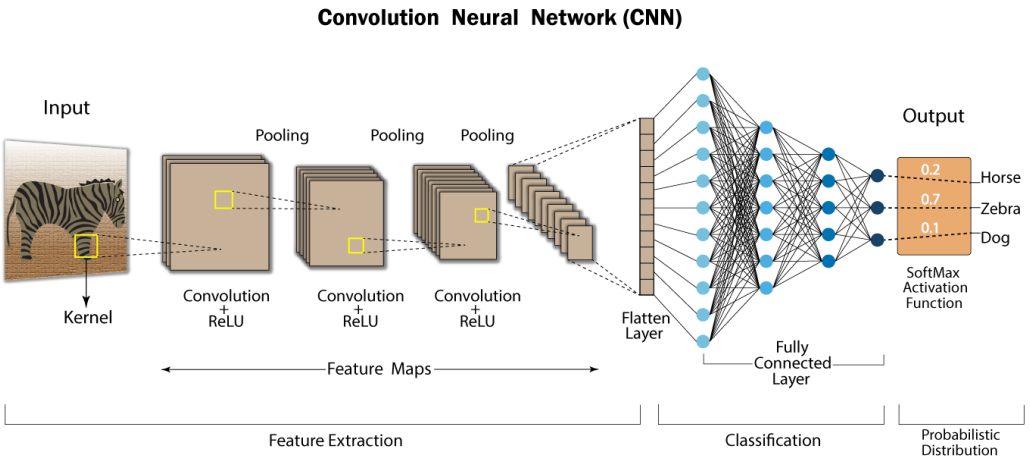


Figure 2.5: Example CNN for animal classification, from [5]

- Stride

  Stride is a parameter which gives the network a chance to even further decrease the number of mapped neurons (often referred to as down-sampling) [7]. In

the previous example, it is not mentioned how exactly the kernel moves across the picture, however, it is somewhat implied that it moves one row/column at a time (based on the calculations and the input size). This assumes some overlap between each of the local regions scanned by the filter. This overlap can be reduced, as well as the size of the output by increasing the stride, which simply is the distance a filter moves (horizontally and vertically) between each scan. Figure 2.6 shows how increasing the stride of a convolution affects the size of the feature map; here it is clear that the bigger the stride, the smaller the feature map (assuming all other parameters stay the same).

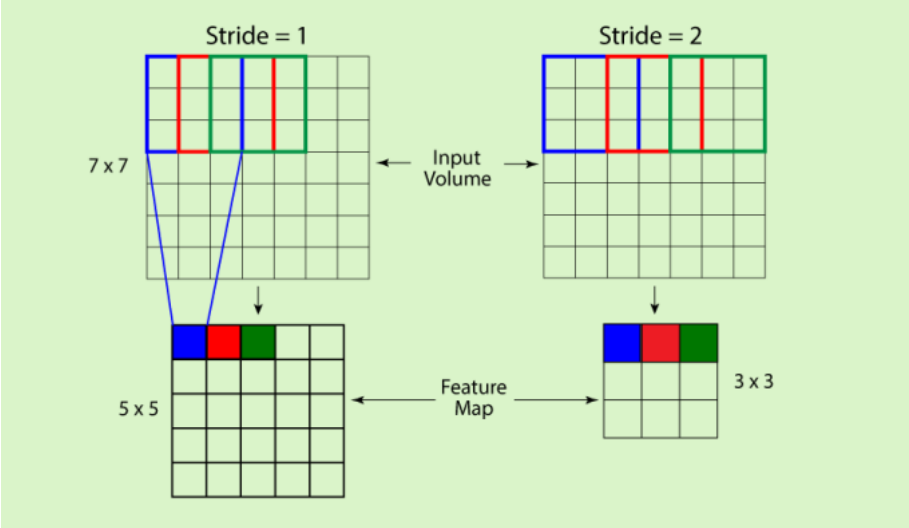


Figure 2.6: Effect of different sized strides , from [6]

- Padding

  Another detail left out in our initial convolution example (which Figure 2.6 reveals) is that when convoluting with a kernel, the dimensions of the feature map will be smaller than what is convoluted [7]. This is fixed by adding a border (padding) around the initial image before the convolution process is started. The formula for padding is: $p(i) = \frac{k-1}{2}$, where p is the needed padding per side of the image, i is the image, and k is the size of the kernel (kernel must be of odd-dimensions). The simplest form for padding is adding zero-values around the outer edge of the picture, however, there do also exist other more sophisticated padding methods.

- Pooling

Pooling is a feature used for down-sampling (reducing complexity of deeper layers) [7]. In Figure 2.8 you can see that pooling in the image processing domain is much like lowering the resolution of an image. One of the most common pooling methods is max-pooling. Max-pooling takes a sub-region of the feature map and takes the max value from that region.

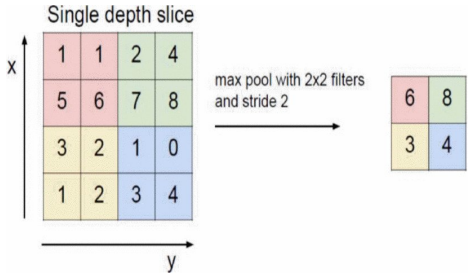


Figure 2.7: Example of max-pooling using a 2x2 filter, from [7]

Since max-pooling is an operation which most often is used for significant down-sampling, the stride of the max-pooling kernel is often equal to the dimension of the kernel - this way the next sub-region will never overlap with the previous. An example of max-pooling can be seen in Figure 2.7.

Other features of a CNN include the use of activation functions and fully-connected layers. The former has been explained previously. The latter is very similar to how nodes are organised in normal neural networks [7]. Each node in the fully connected layer is directly connected with every node in the previous layer. Fully-connected layers are added after the feature extraction and are often the last layers before the output nodes. Activation functions are not unique to CNNs, however, they are equally essential.

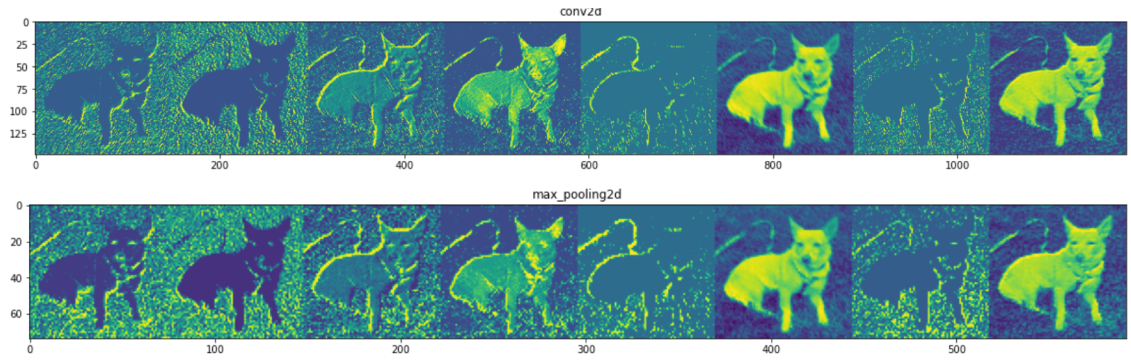


Figure 2.8: Feature maps from convolutions on a dog picture, including max-pooled result, from [8]

2.2.5 Hyperparameters

Hyperparameters refer to the parameters that strictly impact the architecture of the model (e.g. learning rate, choice of optimiser etc.) [23]. With the wide range of resources and technology available today, it is relatively easy to implement and use CNNs [58]. However, in order to get the model to achieve the best possible results, it is necessary to find the optimal hyperparameters. Unfortunately, the large number of hyperparameters combined with the fact that there is no deterministic way to get the best parameters can make this

task rather difficult. Because of this, there have been many different approaches to tuning hyperparameters. In the early stages of CNNs, a lot of tuning was done by guessing and estimating (also referred to as guesstimation). In later years, there have been some attempts at applying different heuristics depending on the problem. For this project, random search hyperparameter tuning was used. This was an extension of the hyperparameter tuning done in previous years. The different hyperparameters that are tuned in this project are described in this section, a list of them is seen below.

- Model Hyperparameters
  - Network Size: This includes kernel size, number of convolutional layers, number of filters and number of nodes in the hidden layers.
  - Dropout Rate
  - Activation Functions
  - Optimisers
- Training Hyperparameters
  - Number of Epochs
  - Learning Rate
  - Batch Size

2.2.5.1 Model Hyperparameters

Network size has roughly doubled in size every 2.4 years due to the constant improvement of memory and computational power over the recent decades [59]. Network size hyperparameters describe features that affect the complexity of the model, which can be measured in the number of weighted connections. The number of convolutional filters, as well as the size of the kernel, are some of the hyperparameters that can impact the network size. The number of dense layers (including their number of nodes), as well as pooling layers, are other hyperparameters that also impact the complexity.

Dropout rate refers to the probability of removing units from the training process [58]. A unit is temporarily discarded from the network, along with all its connections (both incoming and outgoing) [60]. One of the main uses of the dropout layer is to help prevent overfitting. The layer chooses which units to discard based on a random probability, where this probability is the main hyperparameter. This random removal of units results in a ”thinned” network, as it will have fewer overall connections. Furthermore, this makes it impossible to rely on a few correct connections as these might be dropped. Therefore, more connections are ”forced” to learn the correct pattern to solve the task, hence, reducing overfitting.

Activation functions in a neural network are functions that transform the input signal and passes the output signal to the next layer in the network [61]. This output is calculated from the sum of the weighted inputs within each neuron. Not using an activation function would

lead to the output signal being a simple linear function. These are easy to compute, however, they lack complexity and do not have the ability to learn advanced patterns. Hence, activation functions are necessary as long as it is desirable that our neural network works differently than that of a linear regression model. The activation functions experimented with later in this report are all variations of the ReLU activation function.

Optimisers are an important hyperparameter, as there are many different optimisations of gradient descent that exist. Some of these include Nesterov Accelerated Gradient, Momentum, Adagrad, and the previously mentioned Adam and SGD. Where the last two mentions are those optimisers that are compared during hyperparameter tuning on this model.

2.2.5.2 Training Hyperparameters

Number of epochs refers to the number of cycles the model iterates over the training data and performs gradient descent on the weights [23]. Depending on the network, the model may need anything from a few epochs, all the way to many thousands before the neurons have learnt the weight required for the given task.

Learning rate is the amount of which the model descends down the gradient during gradient descent [55]. The learning rate refers to how quickly the model learns, higher learning rates result in big and rapid changes, whilst smaller values result in smaller changes to the weight, often resulting in a need for many epochs [62]. However, too small of a learning rate can result in the model getting stuck at a specific weight or cause the model to never reach the minima. Too big of a learning rate can cause the model to quickly converge to a sub-optimal solution, or do what is called overshoot and make the weight diverge from the minima - this can be seen at the bottom right of Figure 2.9.

Batch size is a parameter unique to mini-batch gradient descent. It refers to the number of training samples randomly chosen for each epoch of the training [23].

2.2.6 Evaluation

2.2.6.1 Metrics

Metrics play a crucial role in obtaining the optimal classifier during training [63]. Therefore, having a range of different metrics is essential to acquiring the optimal model. One of the most useful representations of results for multi-class classification tasks is the confusion matrix [64]. The confusion matrix is a cross table that represents the occurrence between two raters, in classification tasks, these are the true classifications and the predicted classifications. Hence, it is a very useful tool to represent the model's predictive ability. It gives a good indication of areas that need to be improved, whilst also showing where the model performs well.

Some of the most popular metrics are accuracy, recall and precision. Accuracy is a good metric for validating the overall performance of the classifier. Recall evaluates the model's ability to find all the positive units in the dataset and is equal to accuracy when

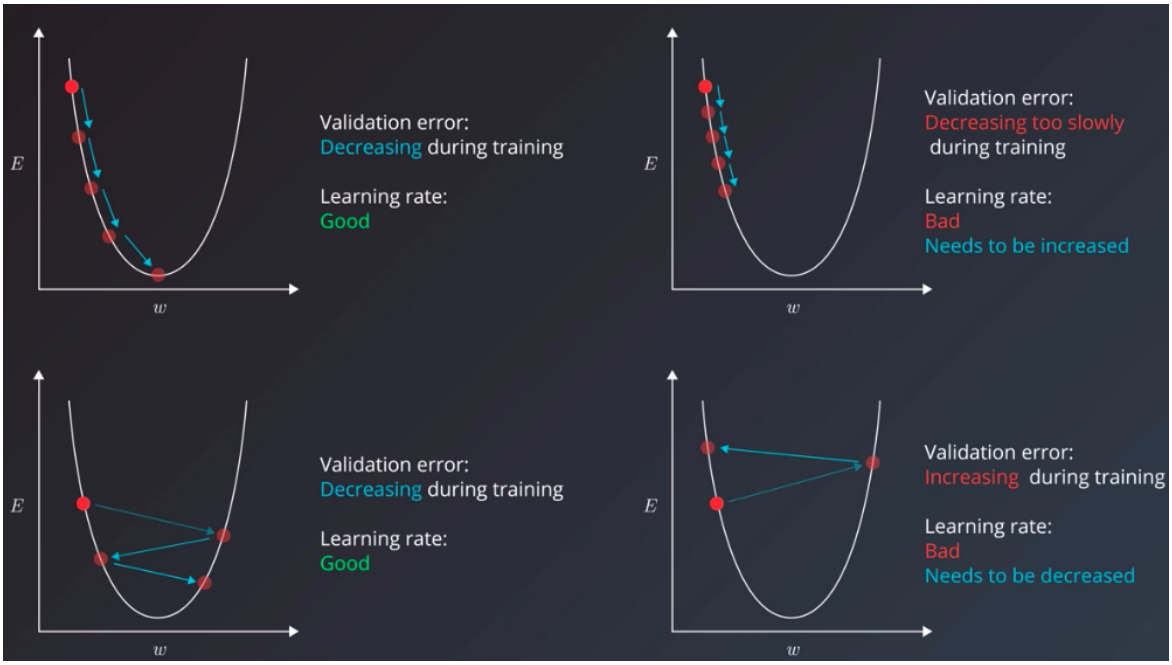


Figure 2.9: Visualisation of good and bad learning rates [9]

dealing with two classes. Precision expresses the proportion of units that the model labels as positive to how many of the retrieved units are actually positive. On their own these metrics can be quite poor; if all classifications are positive (they are all either true positives or false positives) the recall will be 1, whilst if there is only one prediction, and it is correct, precision will be 1. F1-Score is often used to compensate for this. F1-Score is a metric that aggregates precision and recall measures under the concept of harmonic mean - it can be interpreted as the weighted average between precision and recall. The last metric used is logistic loss (often referred to as log loss or cross-entropy loss). Log loss is an indicator of how close the model's prediction probabilities are to the true values [65]. The metrics mentioned and their respective formula can be found in Table 2.1.

2.2.6.2 K-Fold Cross-Validation

Cross-validation is a statistical method used to evaluate learning algorithms [66]. The method splits the data into one part for training, and one part for validation. For k-fold cross-validation, the data is first split into k equally (or close to equal) sized chunks of data, also known as folds. From this k iterations are run, where one of these segments of data is held out as validation data for each iteration, whilst the other k-1 segments are used for training the model, see Figure 2.10. The performance of the model can be found using some pre-determined metrics such as accuracy or f1-score. These results can then be averaged to get an aggregate measure of how well the model performed on the validation data. If k is set equal to the number of training samples in the training set, the cross-validation acts similar to that of the LOOCV evaluation. This is a more tedious form of validation as it creates one model to test per training sample. However, this can be useful in a final evaluation,

Table 2.1: Evaluation Metrics

Metric	**Formula**
Accuracy	$\frac{TP+TN}{TN+TP+FN+FP}$
Recall	$\frac{TP}{TP+FN}$
Precision	$\frac{TP}{TP+FP}$
F1-Score	$2 \cdot (\frac{precision \cdot recall}{precision+recall})$
Log Loss	$H_p(q) = -\frac{1}{N}\sum_{i=1}^{N} y_i \cdot log(p(y_i)) + (1-y_i)) \cdot log(1-p(y_i))$

as each model gets fed more training data and is likely to perform better when averaging the performance. The final validation of the model created for this report is run using an approach named stratified k-fold cross-validation. Stratified k-fold cross-validation forces a class balance throughout the dataset, resulting in training on an equal number of samples from each class, as well as testing on an equal number from each class.

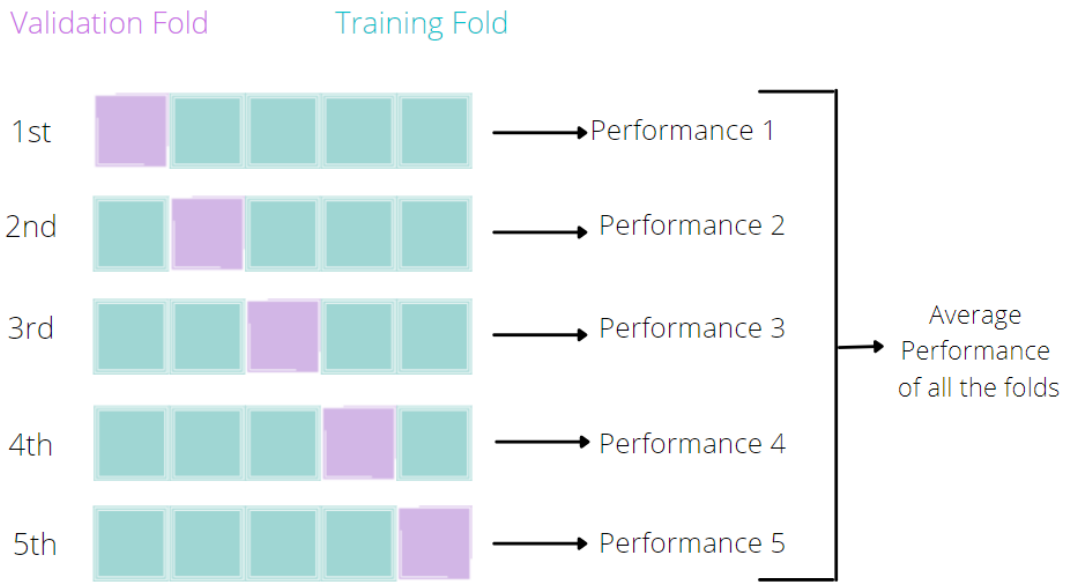


Figure 2.10: K-fold cross-validation, from [10]

b

Chapter 3

Previous Work

3.1 Literature Review

The literature review is split into three sections. Firstly, the data and input of similar ML models will be covered. This includes reviews on the dimensionality of the data used, how the data is segmented, and other relevant steps that are done to the input of the model. The next section covers the different classification methods and how they perform compared to each other. Lastly, the preprocessing steps and slice extraction methods used by Dai [14] and Lackey [15] to create the dataset are described.

3.1.1 Preprocessing - Model Input

The importance of data and its impact on the performance of the model has already been described in subsection 2.2.3. For the preprocessing stage, attempts are made to select the important features of the input, often by removing/reducing unimportant data points. This step is important as it can help speed up the classification process and increase the performance of the classification accuracy [24]. Similar to what is done in this project, all the investigated papers use MRI scans as their main input source. MRI scans are the most extensively used imaging modality in AD research due to their high resolution, non-invasive nature, and moderate cost [12]. How model input is preprocessed in the reviewed studies can be seen below:

3.1.1.1 Dimensionality

The dimensionality of the input data of the investigated models varied considerably. Some models used the whole 3D MRI scan as input [20][21][2], whilst other approaches converted it to either Two-dimensional (2D) [12][22] or in some instances even transformed the data into a one-dimensional feature vector [24]. The preprocessing done to transform the MRI picture into a feature vector is not particularly applicable to the model chosen for this project. This is because the chosen model uses a CNN which can be applied to 2D/3D data, but not on a one-dimensional vector without spacial dependencies. The one-dimensional feature vector approach by Gupta et al. [24] first applied random tree embedding to create a high dimensional transformation of the data, before they reduced the complexity by

using the truncated Singular Value Decomposition (SVD) method. Another method used to lower the input complexity of 3D MRI scans is to extract 2D patches from the different 3D planes [12] [21]. Other studies used the entire 3D data as input, however, due to the enormous amount of extra complexity this brings, the architecture of these networks was much simpler than those that extracted 2D planes [20][21][12][22]. E.g. Valliani and Soni [22] use an 18-layer ResNet with 2D input, on the other hand, Payan and Montana [21] use a network with only one convolutional layer, one pooling layer, and one hidden layer due to the additional complexity given by 3D inputs. For this project, a 2D approach is used, mainly because of its computational feasibility as opposed to the networks using 3D data.

3.1.1.2 Regions of Interest

Multiple studies covering areas such as Regions of Interest (ROI), shape analysis and voxel-based morphometry have found that the amount of atrophy in different regions of the brain (captured by MRI scans), such as the hippocampus, the entorhinal cortex, parahippocampal gyrus and more are very sensitive to the progression from the MCI stage to AD [24]. Furthermore, it has been found that brain topology is not the only good indication of AD - Cerebrospinal Fluid (CSF) biomarkers, as well as Apolipoprotein-E (APOE) genotypes, have been found to also assist in the prediction of AD [24]. The latter because it has been found to contribute to the risk of developing sporadic AD in carriers of the disease. To extract anatomical ROIs several different brain atlases have been used, these include the Automated Anatomical Labeling (AAL) atlas, Harvard-Oxford atlas, and the Brainnetome atlas [24][67]. These atlases include probabilistic mappings of up to 246 segmented regions of the brain [24].

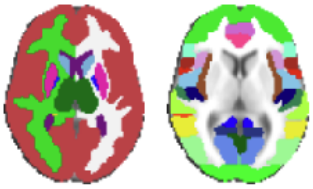


Figure 3.1: Harvard-Oxford cortical and sub-cortical atlases [11]

3.1.1.3 Skull Stripping and Linear Registration

Before any of the aforementioned brain atlases can be applied, it is necessary to make sure that all the brain scans are fitted to an equal template. For a brain to satisfy this criterion it will need to be skull stripped, linearly registered, and if desirable, also segmented. Skull stripping is a preprocessing step which removes everything captured by the MRI which is not the brain itself, this is mainly the skull and surrounding bones. Linear registration is the step of transforming a brain into a standard format such that a template can be applied. Segmentation is the step of separating Gray Matter (GM), White Matter (WM), and CSF from each other - each can then be separately extracted using a template.

Linear registration was the most used preprocessing step as it appeared in all the reviewed studies[24][20][21][12][22][2]. The transformation into a standard format is not only useful for applying templates. The transformation places all brain regions into a standardised space. Therefore, a CNN will be more likely to find patterns in the data, as the local regions captured by the convolutional layer will connect to the same regions in the

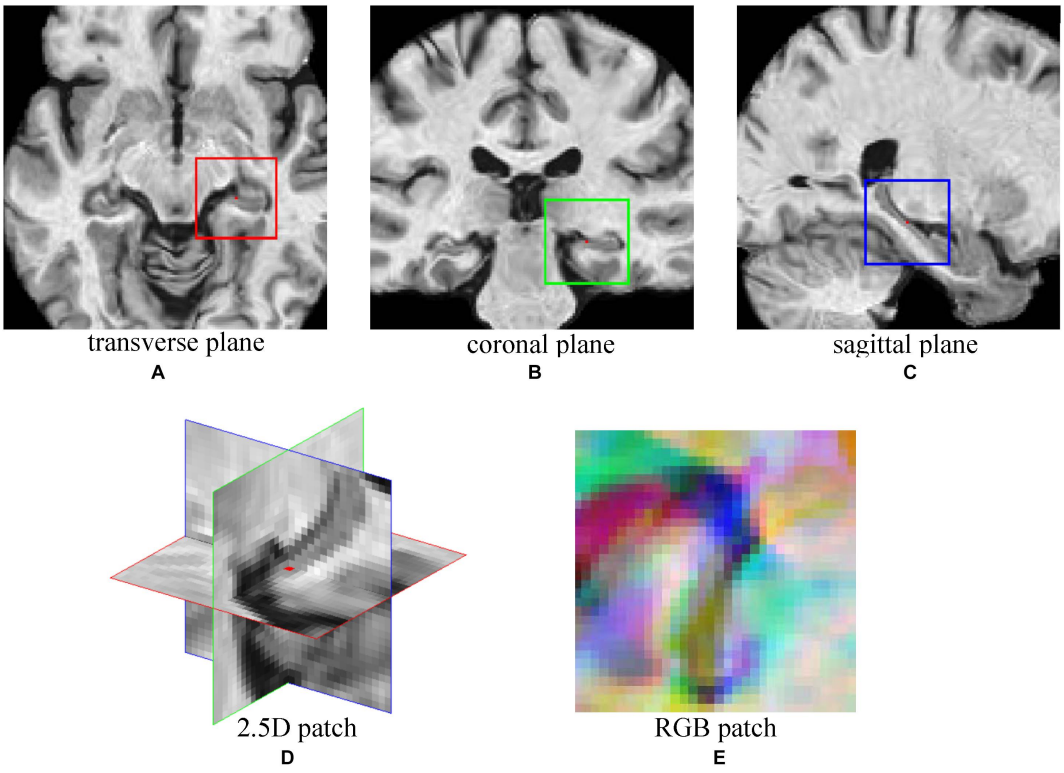


Figure 3.2: 2.5D patch taken from areas around the hippocampus [12]

brain despite differences in the size and shape of the original brain. These transformations are also good to help prevent overfitting [22]. Statistical Parametric Mapping (SPM) and Diffeomorphic Anatomical Registration Exponentiated Lie Algebra (DARTEL) were used [2] to map (or normalise) the brains to the standardised Montreal Neurological Institute (MNI) space. MNI space is one of the most used standardised formats for MRI data - it also appeared in [24] and [12]. Skull-stripping was done in most of the studies with the exception of [21]. Some studies segmented the brain into GM, WM and CSF [2], while others focused on GM and CSF [24].

3.1.1.4 Extraction of 2D Planes

Extraction of 2D planes from 3D data was done by Lin et al. [12] who extracted three 32×32 patches from the transverse, coronal, and sagittal plane - all with centers at the same 3D coordinate. This was then embedded into a 2D RGB patch. The combination of these patches is often referred to as a 2.5D patch, see Figure 3.2 for reference. A similar, but simpler procedure was done by [22] who extracted the median axial slice from a set of 3D MRI scans. The idea of extracting 2D slices from each of the different planes of an MRI scan is a key component to this project, as this project uses a very similar approach to the previously mentioned 2D architecture.

3.1.1.5 Data Augmentation

Multiple data augmentation techniques were introduced in the reviewed studies. One study increased the dataset from 1409 samples to 3000 samples (1000 samples in each class, NC, MCI and AD) by applying deformation, cropping, rotation, flipping and scaling of the different MRI scans [2]. The 2D slice approach used by Lin et al. [12] extracted multiple slices from different locations in the brain to increase training samples. The extraction was random, however, the centre of all extractions had to be in either the left or right hippocampus region (due to its high AD correlation), and one extraction centre had to be at least two voxels away from the centre of any previously extracted planes. This increased the training size by a factor of 151 (151×3 planes extracted per sample).

3.1.2 Classification Methods

The progression within the field of ML and the rise of computational power have led to a big increase in the use of computational classification tools. This is particularly necessary for the detection of AD, as the complex patterns resulting in the disease are yet to be fully understood by experts - hence, advanced models can detect patterns which the field has yet to pick up on. The reviewed studies mainly use CNNs, however, there is also some use of SVMs. These studies are described further below.

3.1.2.1 Support Vector Machines

Gupta et al. [24] applied a kernel-based multiclass SVM classifier with a grid-search approach. The hyperparameters of the model were tuned based on results from running several 10-fold stratified cross-validation experiments. The SVM used a truncated SVD feature vector, this made it easy to incorporate other numerical features such as the volume of GM, CSF biomarkers, and feature values from the APOE genotype.

3.1.2.2 Convolutional Neural Network

Variations of CNNs were the most common approach used for AD classification. Lin et al. [12] used a 2D CNN mainly containing convolutional layers and pooling layers before a 2-output softmax. Valliani and Soni [22] also used 2D inputs, however, they used a pre-trained 18-layer ResNet where each layer defines a 3×3 convolutional layers with the addition of arrows across layers (residuals). A ResNet is a type of CNN which uses shortcut connections to allow input from earlier layers, to be accessible to deeper layers [22]. This allows the network to become ”very deep” making it more accurate and easier to optimise [22].

Payan and Montana [21] implemented both a 2D CNN and a 3D CNN, where the latter performed the best. This implementation was very similar to Vu et al. [20] who not only used MRI scans, but also PET scans as input to their model. Both implementations included an input of a 3D MRI scan, followed by one convolutional layer, one pooling layer, and one fully connected layer. However, Vu et al. also used a 3D PET scan as an input to the network, combining the neurons from the MRI scan and the PET scan when going from the pooling layer to the fully connected layer. The addition of the PET scan increased the

accuracy of the model from 80.62% (only MRI), to 84.72%. These approaches also used sparse autoencoders to extract useful convolutional filters [20]. An autoencoder is a type of neural network which is used to extract features from an image. It has one input layer, one hidden layer, and one output layer. The job of the autoencoder is to reproduce the input, as the output - however, due to the sparsity constraint, the mean activation of all hidden units is very low, hence resulting in each hidden node needing to learn efficient patterns in the data. These learnt weights are then transferred over to the convolutional layers which then get initialised with weights that have been found to represent the data well (instead of initialising the weights randomly). The use of sparse autoencoders increased the accuracy of the PET-MRI model given in [20] from 84.72% to 91.14%. Basaia et al. [2] used a very similar architecture as the previous two methods, however, an autoencoder was not utilised.

3.1.2.3 Model Output

There are several difficult aspects to consider when comparing the result of previous studies. One of these problems is the use of different accuracy measures, e.g. Gupta et al. [24] use 6 different binary classification problems, whilst Payan and Montana [21] simply give the 3-way accuracy score. This can make it hard in some instances to know which one performs better. Another key component is the different dataset sizes. Gupta et al. [24] only had a total of 158 samples, whilst both Payan and Montana [21], and Basaia et al. [2] had well over a thousand samples. However, despite there being some issues with making perfect comparisons between studies, a lot can still be derived from the different approaches, whether it is architecture, hyperparameters, or preprocessing of the data. An overview of the results from the reviewed literature can be found in Table 3.1.

Table 3.1: Summary of reviewed literature

Summary of AD classification results				
Reference	Data	Model Input	Model Type	Accuracy
Lin et al. [12]	ADNI Dataset: 188 AD, 401 MCI, 229 NC	2D 32 $\times$ 32 patches taken from the transverse, coronal, and sagittal plane extracted from original MRI scan	2D CNN with several pooling layers	86.1% accuracy running LOOCV

Gupta et al. [24]	ADNI Dataset: 38 AD, 82 MCI (46 MCIc, 36 MCIs), 38 NC	246 ROIs (210 Cortical, 36 sub-cortical) transformed into a feature vector using random tree embedding and SVD. Feature vector extended with volume of gray matter, 2 APOE genotype features and 3 CSF biomarker features.	SVD using binary classification, classifications run between: AD vs NC, AD vs MCIs, MCIs vs MCIc, NC vs MCIc, AD vs MCIc, and NC vs MCIs	Accuracies ranging from 92.26% to 98.42% depending on the classification task
Vu et al. [20]	ADNI dataset: 145 AD, 172 NC, MCI	3D MRI scans and 3D FDG-PET (18-FluoroDeoxyGlucose PET) scans	Two combined 3D CNNs using binary classification, filter weights acquired using a sparse autoencoder	AD vs NC 91.14% using PET-MRI architecture
Payan and Montana [21]	ADNI dataset: 2265 samples (755 AD, MCI and NC)	$68 \times 95 \times 79$ MRI scans or 68 2D slices from a brain depending on which model is used	One 3D CNN and one 2D CNN, both have pre-trained weights using an autoencoder on patches/segments of the input. 3-way classification.	3-way classification 2D: 85.53% 3D: 89.47%

Basaia et al. [2]	ADNI dataset and Institute of Experimental Neurology (Vita-Salute San Raffaele University, Milan): 407 NC, 280 MCIc, 533 MCIs, and 418 AD	3D MRI scans (some of these being augmented)	3D CNN using binary classification, classifications run between: AD vs NC, AD vs MCIs, MCIs vs MCIc, NC vs MCIc, AD vs MCIc, and NC vs MCIs	Accuracies ranging from 74.9% to 99.2% depending on the classification task
Valliani and Soni [22]	ADNI dataset: 188 AD, 243 MCI, 229 NC	2D Median axial slices from MRI scans (including augmented scans)	18-layer 2D ResNet with two fully connected layers, both binary and 3-way classification	AD vs NC: 81.3%, 3-way: 56.8%

3.2 Implementation and Use of Previous Methods

The data used in this project has been preprocessed by Lackey [15] using methods devised by Dai [14]. This section will describe the methods used, and the decisions taken which led to the preprocessed data.

3.2.1 Data Preprocessing

The 4 main MRI preprocessing steps (and the tools used) as outlined by Dai [14] are found below:

1. Skull Stripping

   The Robust Brain Extraction (ROBEX) tool was utilised for skull stripping. It was compared to the FMRIB Software Library (FSL) brain extraction tool. Not only did ROBEX perform better, but it also does not need hyperparameter tuning. Hence, one can reliably achieve good and accurate skull stripping results using ROBEX, without the need for extensive knowledge of the hyperparameters required for other tools such as the FSL brain extraction tool.

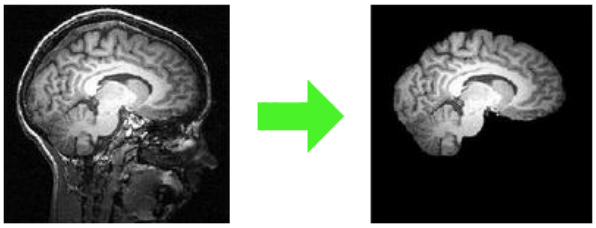


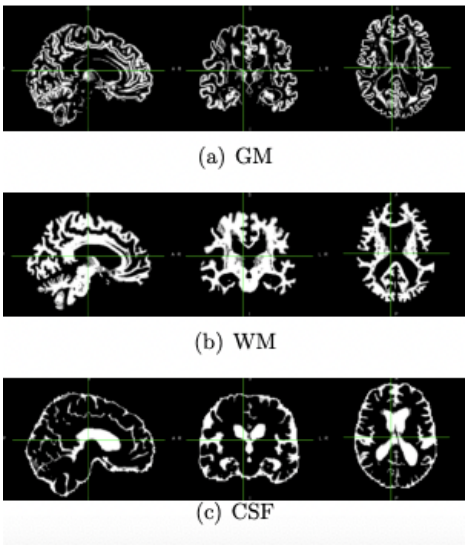
Figure 3.3: Skull extraction example [13]

2. Linear Registration

   Linear registration is the step of transforming a brain scan into a standard template. For this task FLIRT was used, a linear registration tool from FSL. It is the standard tool for affine transformations and was used together with the standard MNI152 template to transform the data.

3. Segmentation

   The substructural atrophy caused by AD stems from the GM found in the brain. This needs to be segmented from the brain scan after the linear registration. The process of segmentation splits an MRI scan into GM, WM, and CSF). FSL-FAST is the tool used for this task. SPM was also considered, however, due to limited accessibility and worse performance it was not used.



(a) GM

(b) WM

(c) CSF

Figure 3.4: Result of segmentation [14]

4. Region of Interest Extraction

   Extracting ROIs requires two steps. (1) Create a mask with the desired ROIs (2) Use the newly created mask to extract ROIs. The tool used for this was FSL-Maths, a highly used graphics calculator. The ROI mask was generated using the Harvard-Oxford cortical and subcortical atlases. Using this mask, seven substructures were selected: Hippocampus, amygdala, caudate, putamen, pallidum, thalamus and mesial temporal lobe. A volume-metric view of this can be seen in Figure 3.5.

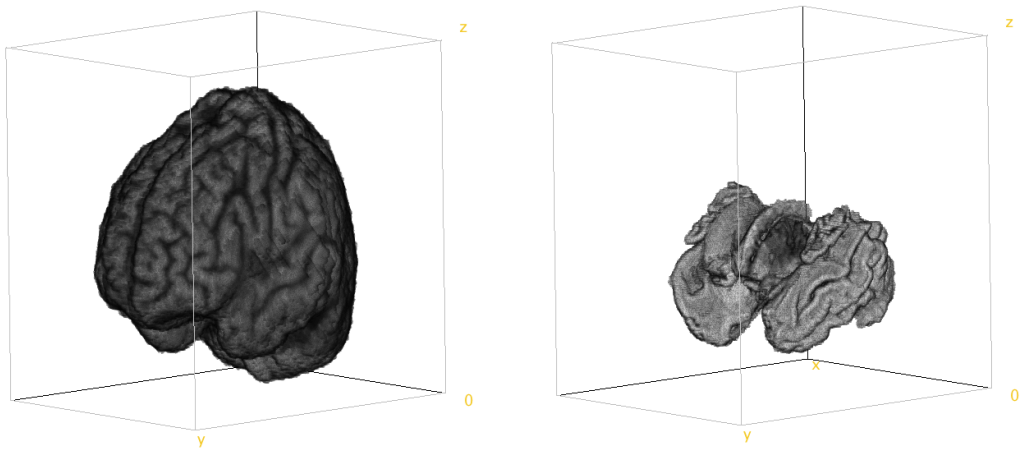


Figure 3.5: Skull stripped and linearly registered brain, before (left) and after (right) ROI extraction

3.2.2 2.5D Slice Extraction

The slice extraction outlined below is from Lackey [15], based on Dai [14].

3.2.2.1 Image size

The next step following the preprocessing of the MRI data was to extract the 2D planes. Lackey [15] found that the slices extracted performed the best when around 160 $\times$ 160 pixels - however, the increased performance from the smaller size of 64 $\times$ 64 was not significant enough. Hence, the size of the slices was decided to be kept as 64 $\times$ 64, which reduced the training time considerably, and allowed for a more complex network.

3.2.2.2 Slice Point

The center of the slices was found to be located at (72, 118, 63) where the coordinates represent where the x-slice, y-slice, and z-slice are performed respectively. The slices with this centre had a smaller loss, better accuracy, and improved MCI class performance than other centers.

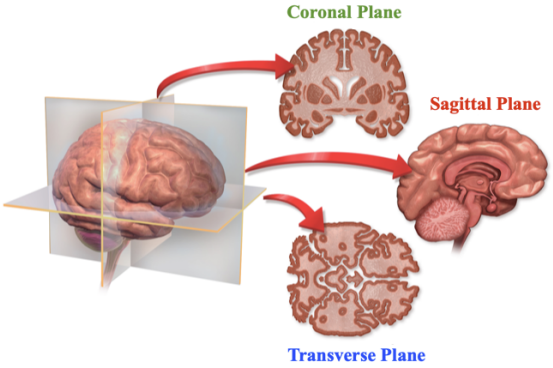


Figure 3.6: Anatomical planes of the brain [14]

3.2.2.3 Plane Size

Dai [14] intended that each of the extracted planes would include an average of multiple slices. E.g. averaging the results from 62-82 on the x-axis would result in an averaged coronal slice. However, Lackey [15] found that doing this muffled the image, and reduced performance. The best performing approach was keeping only one slice for each plane, which is what the final data has.

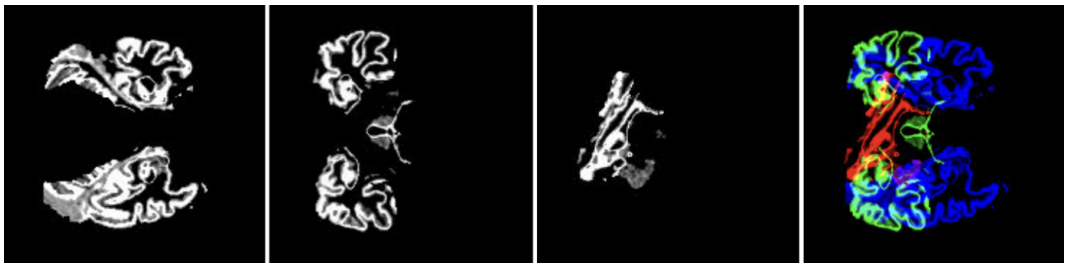


Figure 3.7: 2.5D extracted slices [15]

3.2.3 Data augmentation

Lackey [15] tested multiple ways of varying the data using Keras Image data generator preprocessing library. These tests included uniformly hanging rotation, brightness adjustments, and zoom. The results showed that the base CNN did not improve anything by the use of data augmentation. However, these experiments were again run on the final model (which is the base model for our project) resulting in a slight advantage in increasing the brightness of the input images. v

Chapter 4

Research Methods

4.1 Methods

The project was initially conducted on an Apple MacBook Pro M1 2020, however, the main experiments were quickly shifted over to Google Colab. The experiments were run using Python 3.7 and Tensorflow version 2.8.0. Google Colab has the option to select a specific runtime type. Here GPU was selected, and in most cases, this resulted in experiments being run on a Tesla K80 GPU, the CPU was an Intel(R) Xeon(R) CPU varying from 2.20GHz to 2.30GHz (exact specifications can be seen in Appendix F).

Furthermore, all experiments were run using k-fold validation where $k = 5$. There was a 0.8/0.2 split on the data, where 0.8 was for training, and 0.2 was for testing. The already split training data was then split again, where 0.2 was used as validation data, and the remaining for training.

The dataset used is preprocessed as described in section 3.2. The dataset originally consisted of 1070 brain scans: 303 NC, 524 MCI, and 243 AD. This was then reduced to 729 brain scans, as previous research had proven that class imbalance (different number of training samples per class) negatively impacts the model performance [15]. This left us with 243 samples for each class.

For the experiment, an ensemble of three models is used. Here the result of each of the classifiers is summed together before picking the classified class. The input to each of these models was $64 \times 64 \times 1$, where each input is a slice from either the coronal, transverse, or sagittal plane of the brain. The architecture used for these experiments is based on Lackey [15]. Their architecture can be seen in Figure 4.1, with Table 4.1 explaining the details. The code and model summary can be seen in Appendix D.

The model was tuned using random search where one parameter is varied, and the best performing is kept in the final model. The base model stays the same when switching between hyperparameters.

4.2 Considerations

The hyperparameter search performed does not necessarily obtain the best hyperparameters. There is a chance that two sub-optimal hyperparameters (when looked at separately)

perform better when combined. However, with the number of parameters tweaked it would not be possible to perform a nested hyperparameter search (checking all the combinations). Multiplying the number of values per hyperparameter shows that this would give us over 5 million experiments, which is an unreasonable amount for this project.

Table 4.1: Initial model architecture details

Layer	**Specifications**
Input Layer	Dimensions: $64 \times 64 \times 1$
Convolutional Layer	Dimensions: 64×64 Filters: 32 Kernel Size: 3×3
LeakyReLU	
Convolutional Layer	Dimensions: 64×64 Filters: 64 Kernel Size: 3×3
LeakyReLU	
Max-pooling	Dimensions: 21×21 Kernel Size: 3×3
Convolutional Layer	Dimensions: 21×21 Filters: 256 Kernel Size: 3×3
LeakyReLU	
Max-pooling	Dimensions: 7×7 Kernel Size: 3×3
Convolutional Layer	Dimensions: 7×7 Filters: 256 Kernel Size: 3×3
LeakyReLU	
Batch Normalisation	
Max-pooling	Dimensions: 3×3 Kernel Size: 2×2
Flatten	Number of Nodes: 2304
Dense Layer	Number of Nodes: 410
LeakyReLU	
Dense Layer	Number of Nodes: 410
LeakyReLU	
Dense Layer	Number of Nodes: 410
LeakyReLU	
Dropout Layer	Drop Amount: 0.25
Output Layer	Number of Nodes: 3 Activation Function: Softmax

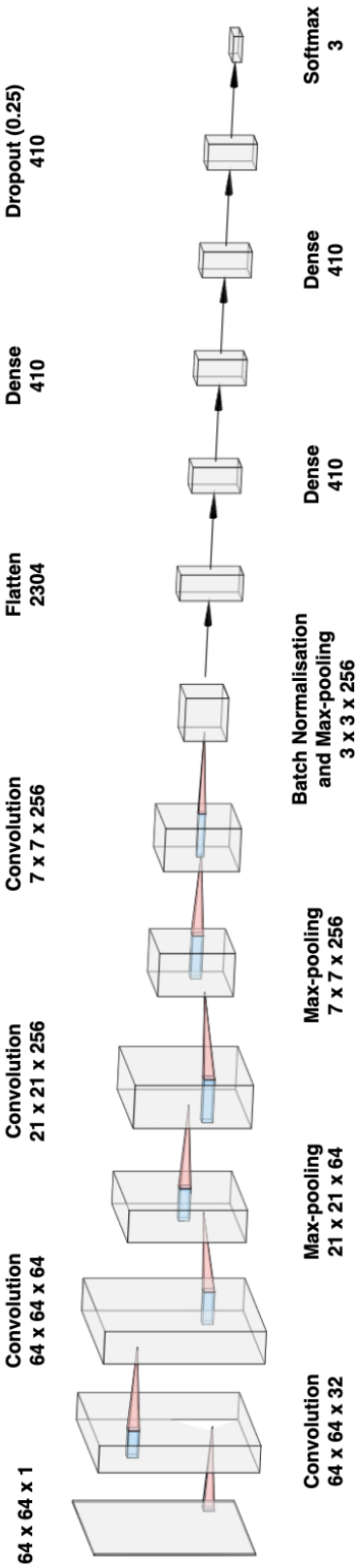


Figure 4.1: Initial model architecture

Chapter 5

Model Construction

This section covers experiments performed to optimise the architecture and structure of the final model and training approach. The section starts by covering hyperparameters relevant to the model itself, before moving on to how the training parameters were chosen.

5.1 Ensemble

An ensemble model uses a combination of predictions from a collection of independently trained models to make one final prediction [68]. It is considered to be a state-of-the-art solution to many machine learning problems as it has the potential to improve overall predictive performance compared to individual models [68]. Research has shown that weighted ensemble models can outperform those ensemble models that are not weighted [69]. In a weighted ensemble, each of the individual models has an associated weight. This means that the models can contribute unequally to the final prediction. On the other hand, in an unweighted ensemble, each model contributes equally to the final prediction. Unweighted ensemble models have been used previously with a good improvement in overall accuracy [15]. This section compares weighted ensemble methods to the non-weighted approach to see which one is preferable (see Figure 5.1 and Figure 5.2 for model overview).

5.2 Ensemble Methods

The method investigated for our weighted ensemble bases itself on the validation accuracy of the model. Validation accuracy is an estimate of the model's true performance. The higher the number of folds during k-fold cross-validation, the more models are created (see Figure 2.10). The assumption is that the more models are created, the closer the averaged validation accuracy will be to the true accuracy. If this holds, it would be possible to weight the separate models on a metric close to their true accuracy.

Three separate evaluation runs were performed to assess this assertion: 5-fold cross-validation, 10-fold cross-validation, and 20-fold cross-validation. Each of these evaluations was run 5 times, and the scores reported in Table 5.1, Table 5.2, and Table 5.3 are the average performances from the 5 runs.

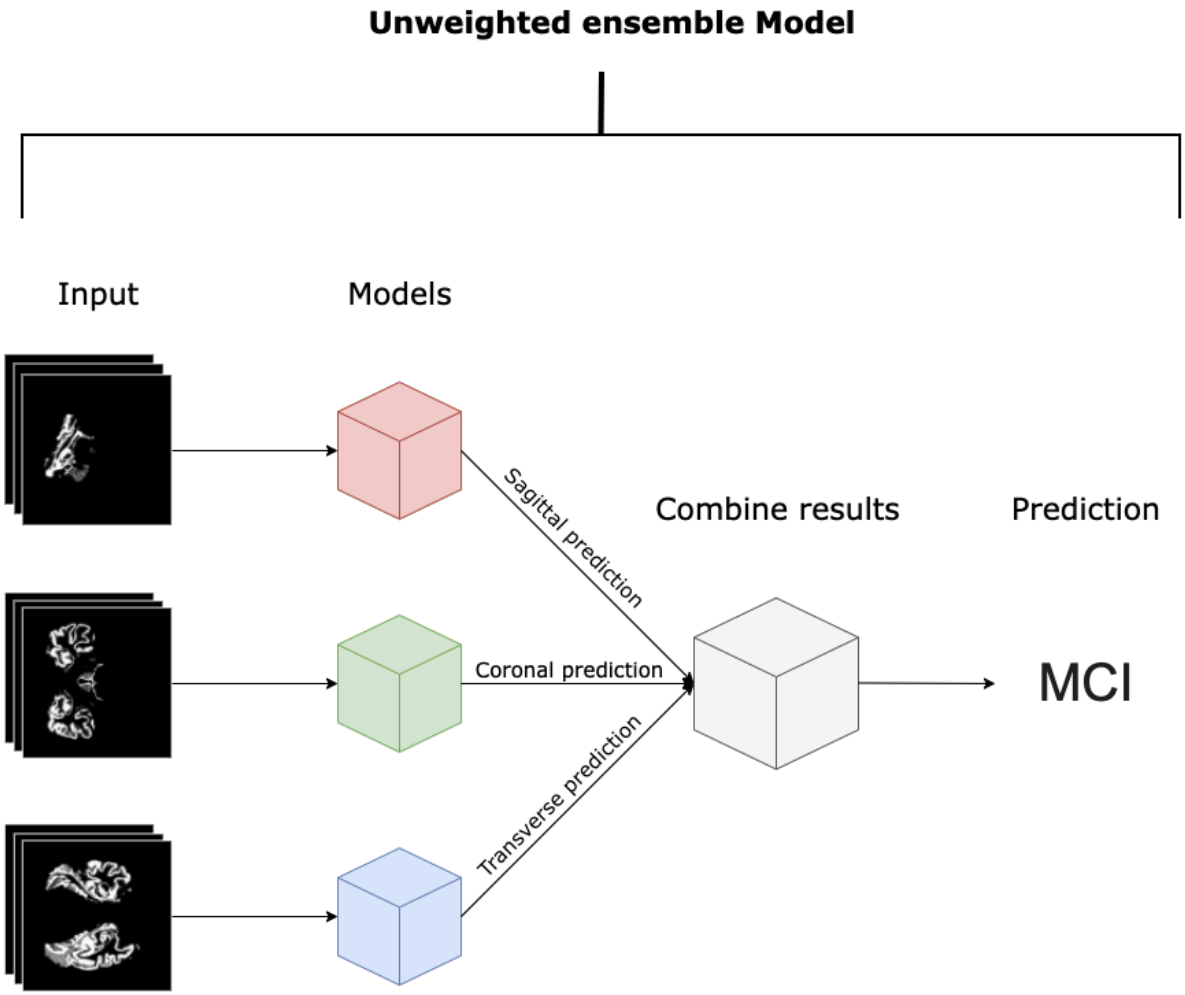


Figure 5.1: Basic unweighted ensemble model overview

These tables show each of the separate model's true accuracy, the validation accuracy and the difference between them, as well as their combined difference. The last two rows shows the performance an ensemble model would have both if weighted and unweighted.

Analysing these tables we can see that the previous assertion is true. The combined average difference between the true accuracy and the validation accuracy lowers as the folds increase.

Additionally, the tables show the effectiveness of ensemble models. In each of the previous experiments, both of the ensemble methods outperform each of the individual plane models. This implies that each of the models does bring some information unique to that plane.

For the weighted ensemble method, the validation accuracy for each model is multiplied with their respective prediction before it is summed together (combined). Following this, the highest value is picked. The two ensemble equations are given in Equation 5.1 and Equation 5.2.

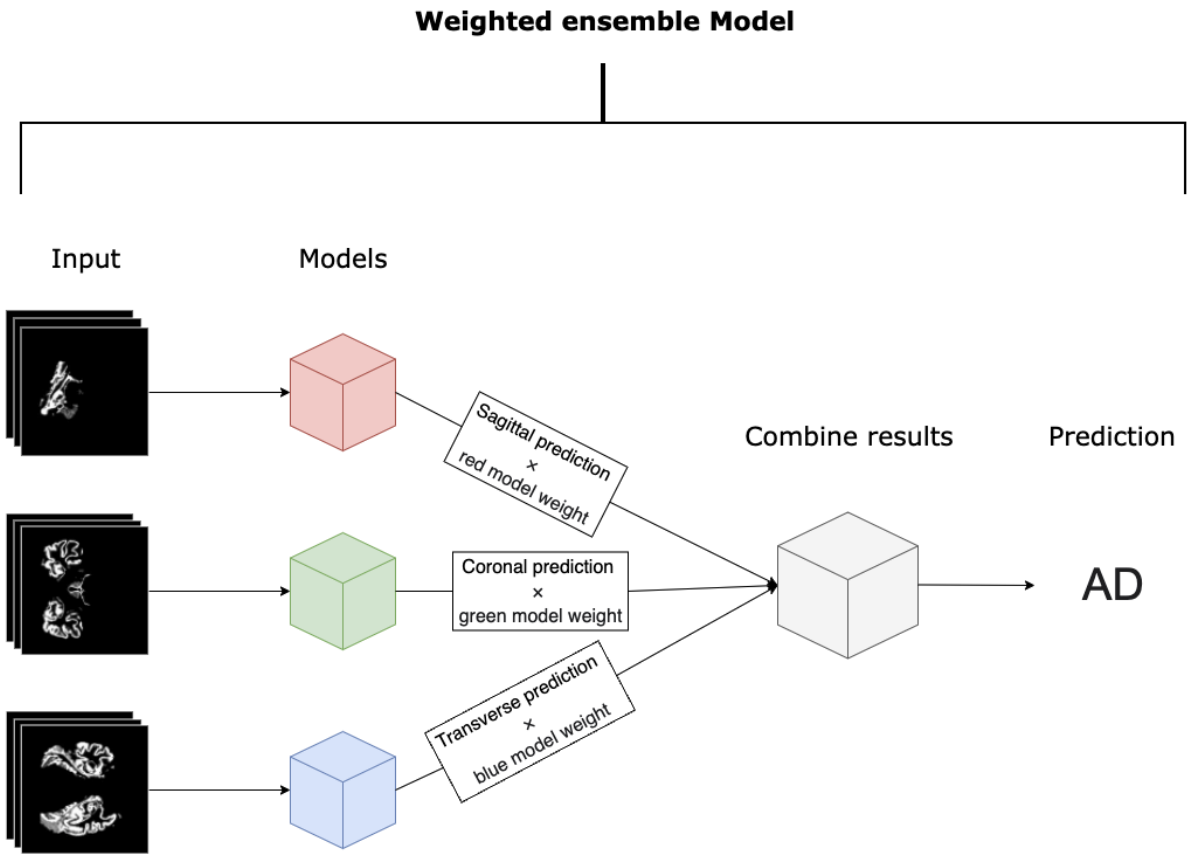


Figure 5.2: Basic weighted ensemble model overview

Unweighted ensemble:

$$y = \underset{c_i \in C}{\operatorname{argmax}} \sum_{m_j \in M} P(c_i | m_j, L) \tag{5.1}$$

Weighted ensemble:

$$y = \underset{c_i \in C}{\operatorname{argmax}} \sum_{m_j \in M} P(c_i | m_j, L) \cdot v_{m_j} \tag{5.2}$$

where y is all the resulting predictions, C is the set of all classes, L is the set of all samples, M is the set of all models, $P(c_i|m_j,L)$ returns the multi-class predictions from model j on the set of samples L, v_{m_j} refers to the overall acquired validation accuracy of model j.

Furthermore, the weighted method applies the validation accuracy to each prediction after all predictions have been made. Applying the validation accuracy means that the prediction made by a model is multiplied by the average validation accuracy of that model, as acquired from all the folds in the k-fold cross-validation. Hence, the weight of each model is the average validation accuracy of that model.

Table 5.1: 5×5-Fold experiments on ensemble methods

5×5-Fold	Red	Green	Blue
Mean True Accuracy	0.6938441	0.7426509	0.7006802
Mean Validation Accuracy	0.7063248	0.7326495	0.7052991
Absolute Difference	0.0124806	0.0100013	0.0046189
Combined Mean Difference	0.0090336574487020		
Average Ensemble Accuracy (Unweighted)	0.7607000000000001		
Average Ensemble Accuracy (Weighted)	0.7610999999999999		

Table 5.2: 5×10-Fold experiments on ensemble methods

5×10-Fold	Red	Green	Blue
Mean True Accuracy	0.7253196	0.7552625	0.7277815
Mean Validation Accuracy	0.7286363	0.7530303	0.7180303
Absolute Difference	0.0033167	0.0022322	0.0097512
Combined Mean Difference	0.00510		
Average Ensemble Accuracy (Unweighted)	0.78472		
Average Ensemble Accuracy (Weighted)	0.78526		

The most important result from these experiments is that the more representative the validation accuracy is of the true accuracy, the better the weighted ensemble method works as opposed to the unweighted method. Whilst the increase is just over 0.1% at 20-folds, it seems to still be increasing as the number of folds is increased. Hence, with n=243 it is likely to gain a few tenths of a percentage purely using the weighted method over the unweighted method. Therefore, we use the weighted ensemble method for the evaluation of this project.

Table 5.3: 5×20-Fold experiments on ensemble methods

5×20-Fold	Red	Green	Blue
Mean True Accuracy	0.7425225	0.7778828	0.7363213
Mean Validation Accuracy	0.7413669	0.7744604	0.7418705
Absolute Difference	0.0011556	0.0034224	0.0055491
Combined Mean Difference	0.00337		
Average Ensemble Accuracy (Unweighted)	0.80252		
Average Ensemble Accuracy (Weighted)	0.80366		

5.3 Model Hyperparameters

5.3.1 Random Search Model Optimisation

To optimise the model hyperparameters, random search optimisation was conducted on the unweighted ensemble of 3 models using the CNN seen in Figure 4.1 and Table 4.1. Due to the number of hyperparameters evaluated, random search is the only reasonable approach – as doing nested hyperparameter evaluation would create an unreasonable number of experiments. The overview of all the tuned hyperparameters can be found in Table 5.4. Convolutional layers, kernel size, activation function, and dropout rate are some of the model hyperparameters experimented with. As for algorithm hyperparameters, learning rate, number of epochs, and batch size are also varied.

Table 5.4: Hyperparameter overview

Hyperparameter	Values	Default
Kernel Size	3, 5, 7	3
Activation Function	ReLU, Leaky ReLU, ReLU6, SeLU	ReLU
Additional Conv Layers	0, 1, 2	0
Size of Added Conv Layers	64, 128	N/A
Dropout Rate	0 to 0.5 in strides of 0.05	0.25
Units per Dense Layer	300 to 500 in strides of 20	400
Optimiser	SGD, Adam	Adam
Learning Rate	0.00001, 0.00005, 0.0001, 0.0005, 0.001	0.00005
Epochs	15 to 50 in strides of 5	20
Batch Size	32 to 256 in strides of 32	32

Each experiment is evaluated using 5-fold cross-validation, gathering metrics from each of the individual models, as well as their ensemble model performance. Each of the models are using the same architecture, and this one architecture is what is tuned. This leaves room for future improvements, as hyperparameter optimisation could potentially be done

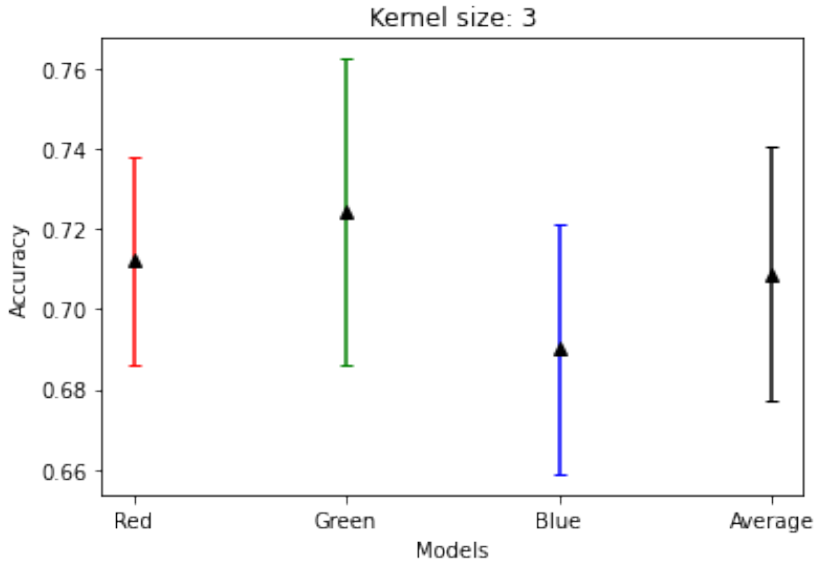


Figure 5.3: Validation accuracy (triangles) and standard deviation (lines) of the separate and combined models when kernel size is set to 3.

on an individual plane level - using 3 different architectures. Each model gathers some unique data, so the probability of one architecture being "perfect" for all three is unlikely – however, a search like this could end up being extremely tedious and might not be beneficial for the results it could produce.

5.3.2 Kernel Size

The test accuracies of each of the different kernel sizes can be seen in Figure 5.3 - Figure 5.6. It shows each of the models' performances, as well as the average accuracy.

As these results show, the different sizes of kernels make a minimal difference. The 3×3 kernel had the best performance with an average validation accuracy of 0.712, which was close to the 7×7 kernel which had a validation accuracy of 0.710. The 5×5 kernel is only slightly behind that with its 0.708. However, these values are so close that if we ran these experiments multiple times they could at certain points pass one another. Because of this lack of impact, it can be useful to look at other metrics to choose kernel size. Due to the size increase of the kernel, each convolution also requires more computational power. The model using a 3×3 kernel spent 129.8 seconds for training, followed by the 5×5 kernel that spent 171.2 seconds, with the 7×7 kernel spending the most time with 250.4 seconds. From this, we can see that the 3×3 kernel is almost twice as fast as the 7×7 kernel when training the model. Thus, because of better efficiency, the 3×3 is used in our final model.

5.3.3 Activation Function

Experiments done by Lackey [15] showed that the ReLU activation function outperformed both the Sigmoid activation function and the hyperbolic tangent function, also referred to

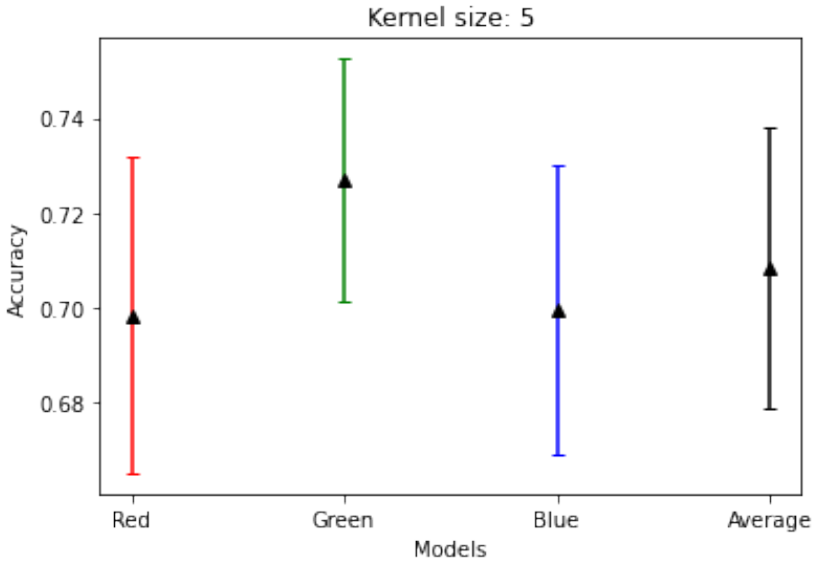


Figure 5.4: Validation accuracy (triangles) and standard deviation (lines) of the separate and combined models when kernel size is set to 5.

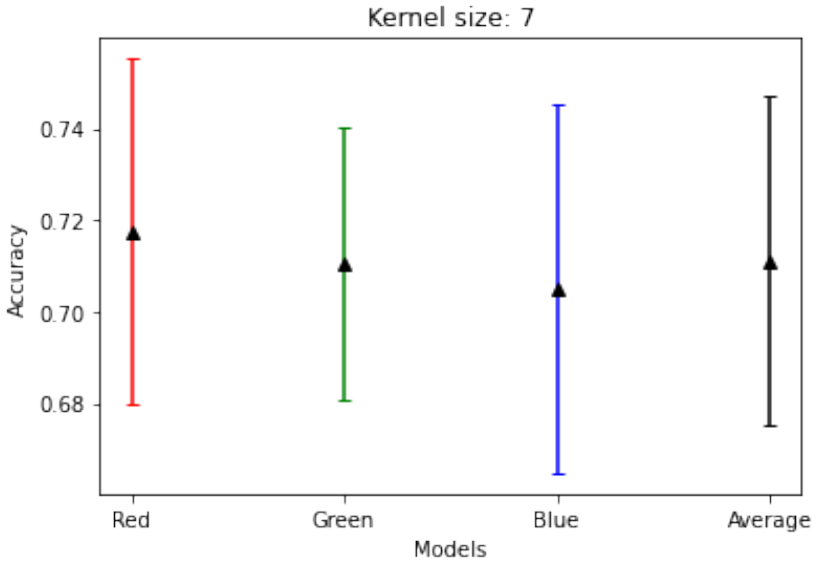


Figure 5.5: Validation accuracy (triangles) and standard deviation (lines) of the separate and combined models when kernel size is set to 7.

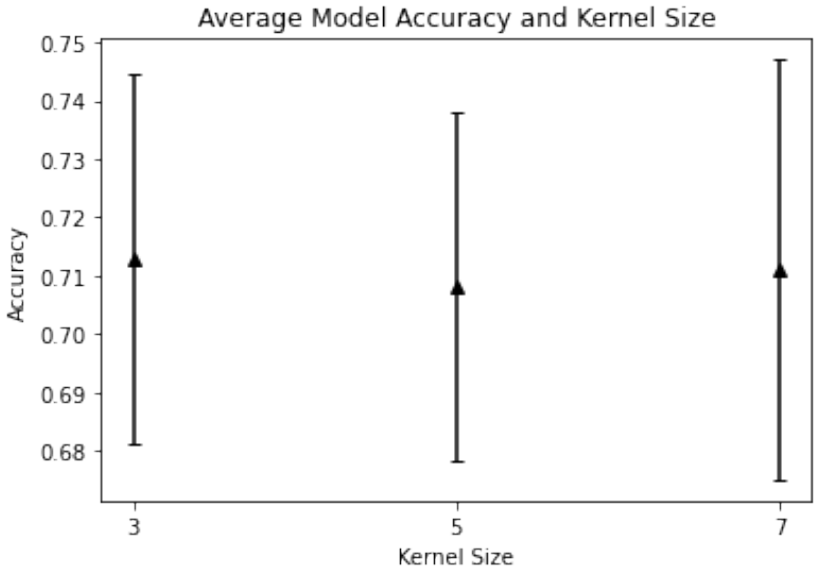


Figure 5.6: Validation accuracy (triangles) and standard deviation (lines) of the ensemble model depending on kernel size.

as Tanh. Hence, experiments done here is exploring variations of the ReLU, and compare these to each other. The following activation functions were explored: ReLU, LeakyReLU, ReLU6, and SeLU.

As seen in Figure 5.7, ReLU6 and SeLU are the weakest performing activation functions. ReLU6 is simply a modification of the normal ReLU function where the activation has a maximum value of 6. The best performing is the LeakyReLU with a 72.7% mean accuracy, followed by ReLU with a 71.6% mean accuracy. The most surprising of these results is the difference between LeakyReLU and SeLU. SeLU is similar to LeakyReLU in that they both get a negative value when $x < 0$, something which is not the case with ReLU and ReLU6. The main difference is that the LeakyReLU is linear when $x < 0$, as opposed to the SeLU which is exponential (see Figure 5.8). The reason behind this difference in performance is unknown, however, it might be worth exploring further in later research.

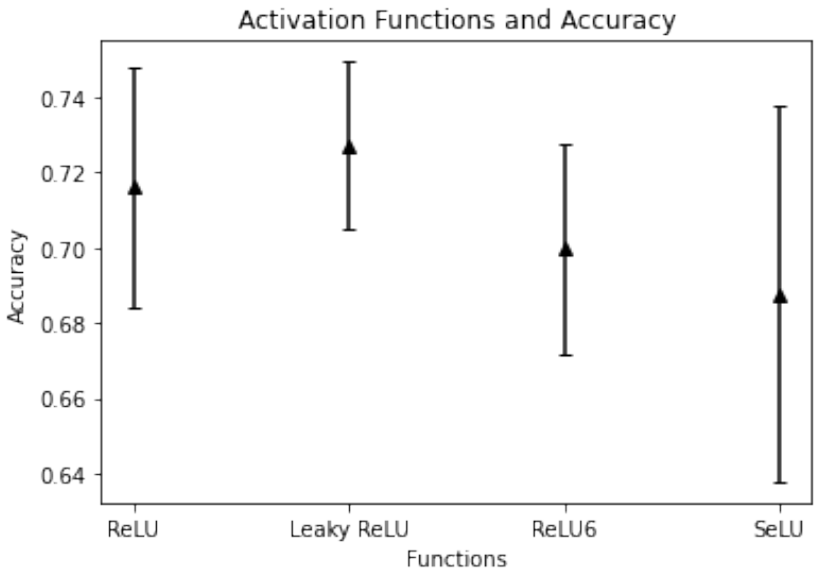


Figure 5.7: Activation functions accuracy

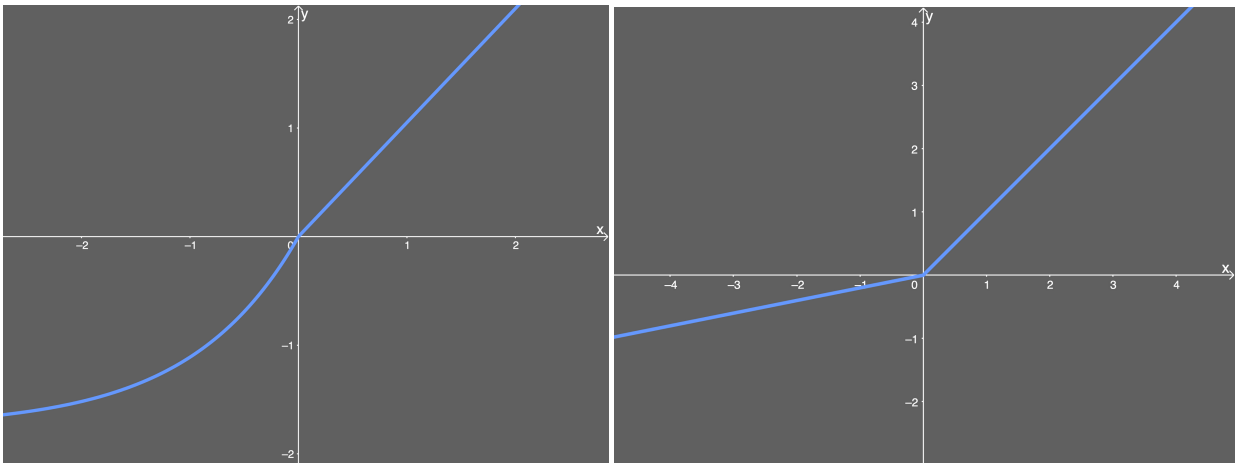


Figure 5.8: SeLU (left) and LeakyReLU (right) [16]

5.3.4 Additional Convolutional Layers

The base model architecture uses 4 2D convolutional layers – one layer with 32 units, one with 64 units, and two with 256 units (in this order). Each layer uses the ReLU activation function. The same activation function is used for all the additional layers tested. The additional layer(s) is put between the 2nd and 3rd convolutional layers (64 units and 256 units respectively).

Table 5.5: Results of extra convolutional layers

Additional Layers, Units	Accuracy	Standard Deviation	Runtime (seconds)
0 Extra Layers	0.70282	0.03744	93.59
1 Extra Layers, 64 Units	0.71235	0.03640	97.73
1 Extra Layers, 128 Units	0.73023	0.02818	100.11
2 Extra Layers, 64 Units	0.71603	0.02077	99.41
2 Extra Layers, 128 Units	0.72062	0.03248	107.09

After running 5 experiments, the one using one extra layer with 128 units proved superior. The deeper the network is, the more computationally expensive it is. Hence, adding one extra layer with 128 units increased the training time by around 9% – from 93.5 seconds to 100.1 seconds. Due to the significant increase in performance, it is clear that adding another layer can prove beneficial to the network – especially when the increase in runtime is this low.

5.3.5 Dropout Rate

Our model architecture has one dropout layer just before our output layer as seen in Appendix D. The default value for this layer is 0.25. To find the optimal value the dropout rate was varied between (and including) 0 and 0.5, with strides of 0.05.

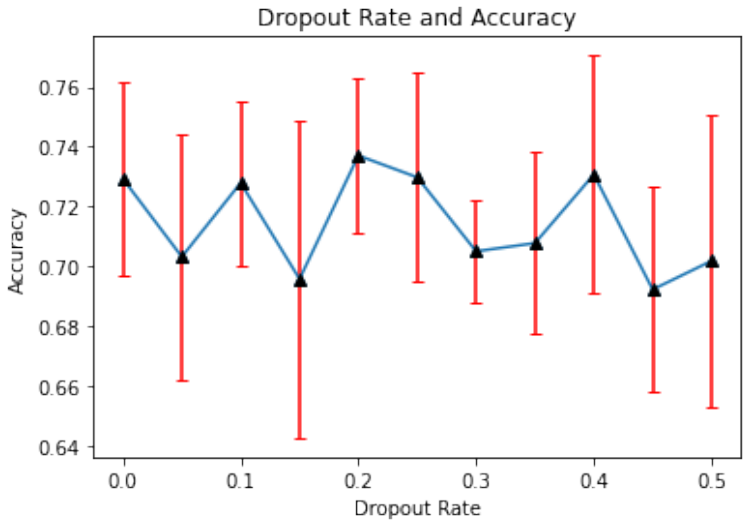


Figure 5.9: Dropout rate and validation accuracy correlation

From Figure 5.9, we can see that the best performing value was 0.20, with a mean validation accuracy of 73.8%. Both 0.25 and 0.40 had accuracies of 73%, however, these values had a higher standard deviation (3.5% and 3.9% respectively) than the dropout rate of 0.20 (2.5%). This implies that their results despite being second and third best, are both less accurate and less reliable than the dropout rate of 0.20.

5.3.6 Units per Dense Layer

For the units (or nodes) per dense layer, the values are changed similarly to that of subsection 5.3.5. The model has 3 dense layers, with a default of 400 units per layer. The result of varying this between 300 and 500 in strides of 20 can be seen in Figure 5.10.

Figure 5.10: Number of hidden units per dense layer and aalidation accuracy correlation

The best performing number of units per layer was 460, reaching a validation accuracy of 74.0% (see Figure 5.10). However, as previously mentioned - the deeper the network, the more time and computational power is consumed by the training. Figure 5.11 presents the correlation between the dense units and the time.

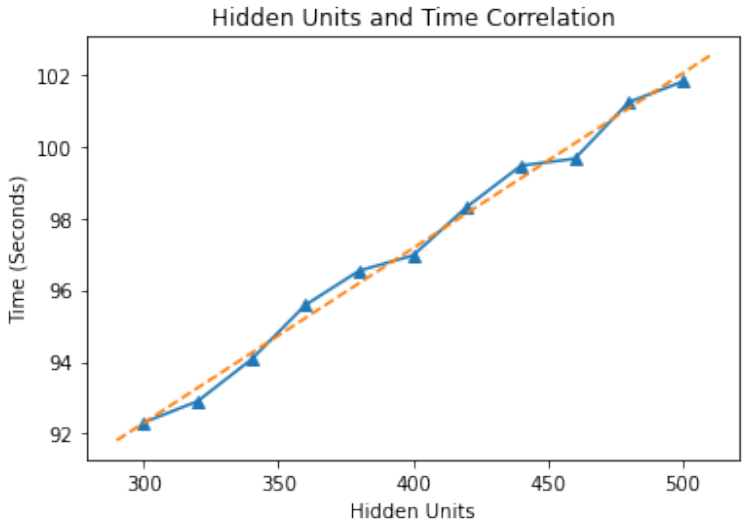


Figure 5.11: Number of hidden units per dense layer and time correlation

The relationship between hidden units and time is very linear. The orange line in Figure 5.11 is a polynomial regression model fit using the method of least squares. The line represents the data points well. The slope of the orange line is 0.0487 – this means that per unit added to each of the dense layers, it takes the training 0.0487 seconds longer to execute. Multiplying the slope with our stride gives us $20 \cdot 0.0487 = 0.974 \approx 1$ second. Hence, a good approximation for this experiment is that for every 20 units added to the dense layers, the training time is increased by 1 second. - which by looking at the graph is just over a 1% increase on average. The only number of hidden units with good results, and less than that of 460 units, is 380. With this number of hidden units, a validation accuracy of 72.6% was achieved. However, despite the approximately 3% more training time required by 460 units per layer, its increased accuracy makes it the chosen hyperparameter value for the model.

5.3.7 Optimiser

For optimisers, we went through SGD and the Adam optimiser. It became clear that the architecture and default parameters were created around using Adam as the optimiser. Adam gave an average validation accuracy of around 71%, whilst SGD gave a validation accuracy of barely over 38% as seen in Table 5.6. Adam is less locally unstable than SGD, however, the outcome seems to mainly stem from the architecture being optimised towards one of the optimisers. Therefore, the choice of optimiser for the model is Adam.

Table 5.6: Results of optimiser experiments

Optimiser Comparison	Validation Accuracy	Standard Deviation
Adam	0.7087797197291765	0.03001544926124008
SGD	0.3813352227995590	0.02327086251759329

5.3.8 Training Hyperparameters

This section covers the experiments run on training hyperparameters, which include: learning rate, batch size, and the number of epochs. Batch size performed the best with its default value of 32. Since the validation accuracy of this test was decreasing as batch size increased for all experiments, an additional test of batch size equal to 16 was done as well. This was to make sure that we did not miss any performance on the lower end of the batch size spectrum. However, the result of batch size 16 ended up with a validation accuracy of 72.7%, which is worse than the 73.4% acquired by batch size 32.

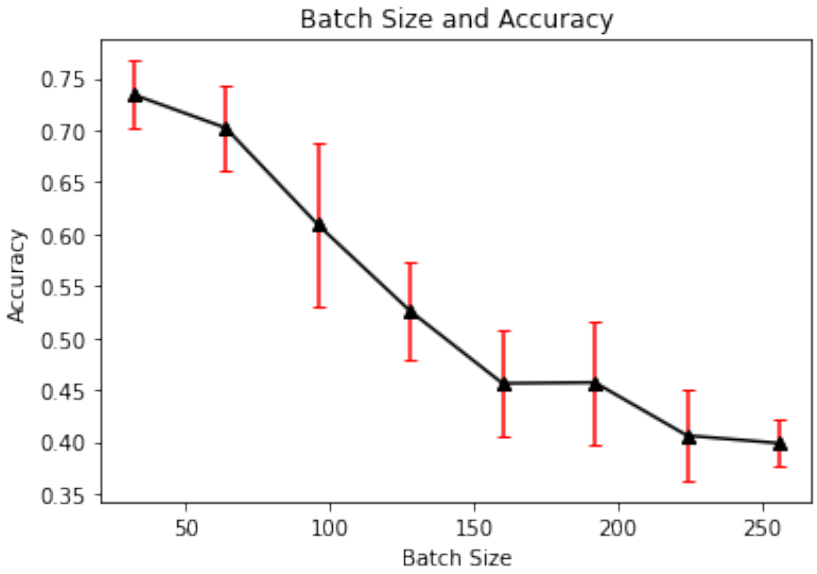


Figure 5.12: Validation accuracy from varying batch size

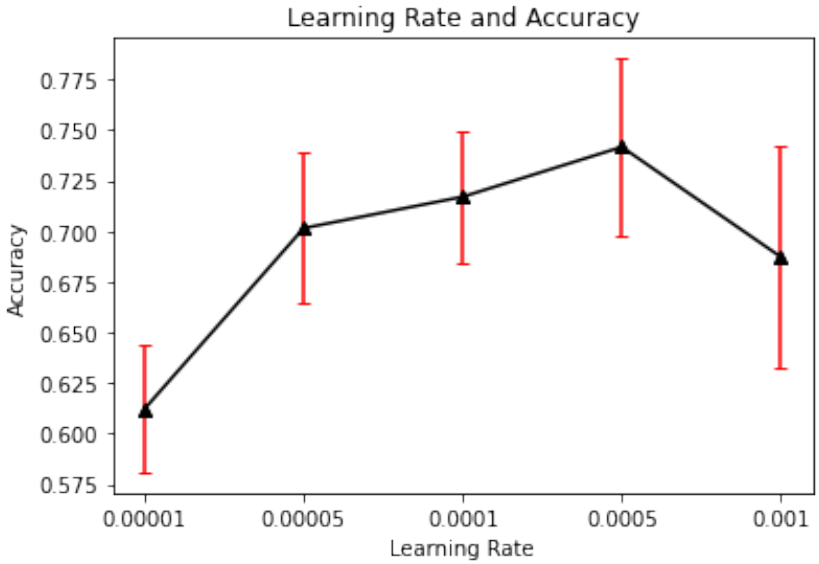


Figure 5.13: Validation accuracy from varying learning rate

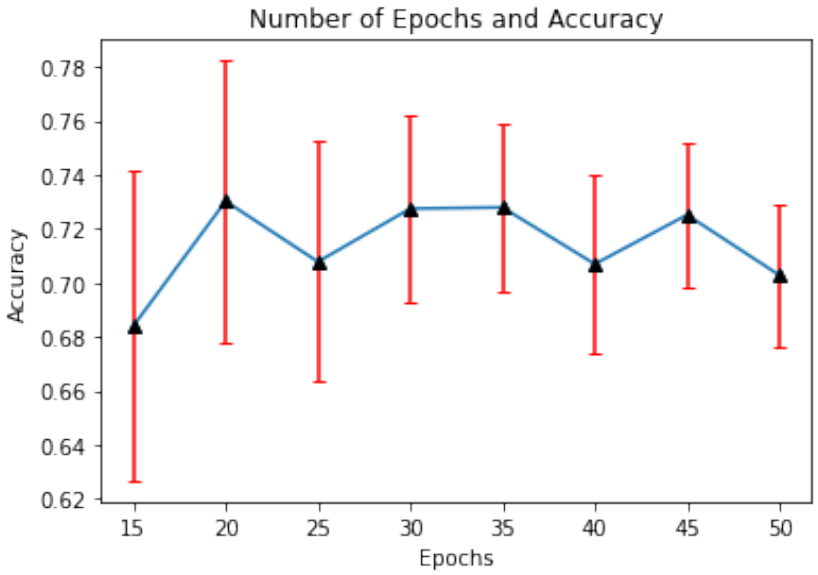


Figure 5.14: Validation accuracy from varying number of epochs

For the learning rate, the experiments proved that the best value was 0.0005 with a validation accuracy of 74.1%. However, the epochs were slightly closer. When the number of epochs is 30 and 35 we get validation accuracies of 72.7% and 72.8% respectively. However, with 20 epochs we get a slightly higher peak at 73.1%. Because of this, we pick 20 as the number of epochs used. However, the results here are so close to each other that the likelihood of this fluctuating from experiment to experiment is high. Hence, good performance on a relatively low number of epochs is promising, as it reduces unnecessary training time.

5.3.9 Final Hyperparameters

Table 5.7 shows the result summary of all previous hyperparameter experiments. The right column shows the hyperparameters that resulted in the best performance. The final model architecture can be seen in Figure 5.15, with model details in Table 5.8.

Table 5.7: Optimal hyperparameter values from hyperparameter tuning

Hyperparameter	Optimal Value
Kernel Size	3
Activation Function	Leaky ReLU
Additional Conv Layers	1
Size of Added Conv Layers	128
Dropout Rate	0.20
Units per Dense Layer	460
Optimiser	Adam
Learning Rate	0.0005
Epochs	20
Batch Size	32

Table 5.8: Final model architecture details

Layer	**Specifications**
Input Layer	Dimensions: $64 \times 64 \times 1$
Convolutional Layer	Dimensions: 64×64 Filters: 32 Kernel Size: 3×3
LeakyReLU	
Convolutional Layer	Dimensions: 64×64 Filters: 64 Kernel Size: 3×3
LeakyReLU	
Max-pooling	Dimensions: 21×21 Kernel Size: 3×3
Convolutional Layer	Dimensions: 21×21 Filters: 128 Kernel Size: 3×3
LeakyReLU	
Convolutional Layer	Dimensions: 21×21 Filters: 256 Kernel Size: 3×3
LeakyReLU	
Max-pooling	Dimensions: 7×7 Kernel Size: 3×3
Convolutional Layer	Dimensions: 7×7 Filters: 256 Kernel Size: 3×3
LeakyReLU	
Batch Normalisation	
Max-pooling	Dimensions: 3×3 Kernel Size: 2×2
Flatten	Number of Nodes: 2304
Dense Layer	Number of Nodes: 460
LeakyReLU	
Dense Layer	Number of Nodes: 460
LeakyReLU	
Dense Layer	Number of Nodes: 460
LeakyReLU	
Dropout Layer	Drop Amount: 0.20
Output Layer	Number of Nodes: 3 Activation Function: Softmax

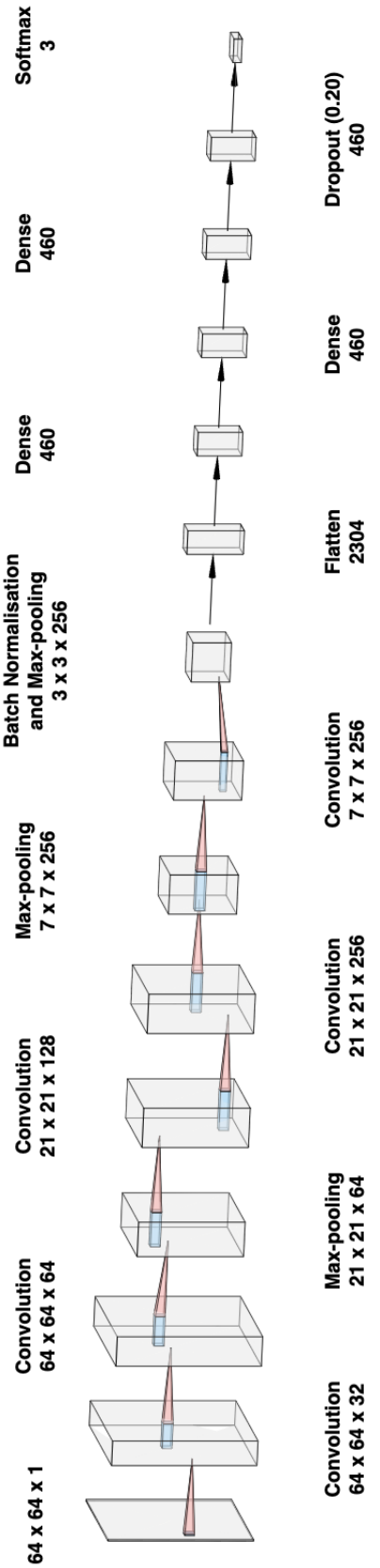


Figure 5.15: Final model architecture

Chapter 6

Results and Discussion

6.1 Results

For the final evaluation of the model, stratified k-fold cross-validation was used with the k-value set to the highest possible value given the size of the dataset. For each fold, the model is trained on 726 samples (242×3) and tested on 3 samples (1×3), where each test set contains one picture from each class. This results in a version of LOOCV, where one sample from each class is left out for validation per fold. Of the test set, 80% were used for testing, whilst 20% of the set was used for validation. The validation set is used to gather the validation accuracy of each model per fold. These accuracies are used to extract the mean validation accuracy which is used to weight the separate model predictions before an ensemble prediction is made (see Equation 5.2).

To avoid over-fitting towards either of the classes, the model is trained on an equal amount of samples from each of the classes. Hence, performance can be dependent on which samples are selected from the NC and MCI classes – as both these sets have more than 243 samples. The end result is 243 ensemble models. Each ensemble model is created by the combination of three models, each of these trained and optimised for one of the three anatomical planes (axial, sagittal, and coronal planes).

6.2 Training

Figure 6.1, Figure 6.3, and Figure 6.5 show the training accuracy and validation accuracy from a randomly selected fold. Figure 6.2, Figure 6.4, and Figure 6.6 show the respective training and validation loss of the aforementioned accuracies. These graph shows that the validation accuracy of the model plateaus much earlier than the training accuracy which is approaching 1. Furthermore, there is a reasonably big gap between the training and validation accuracy, as well as the training and validation loss. However, this is to be expected as the model overfits ons the training data.

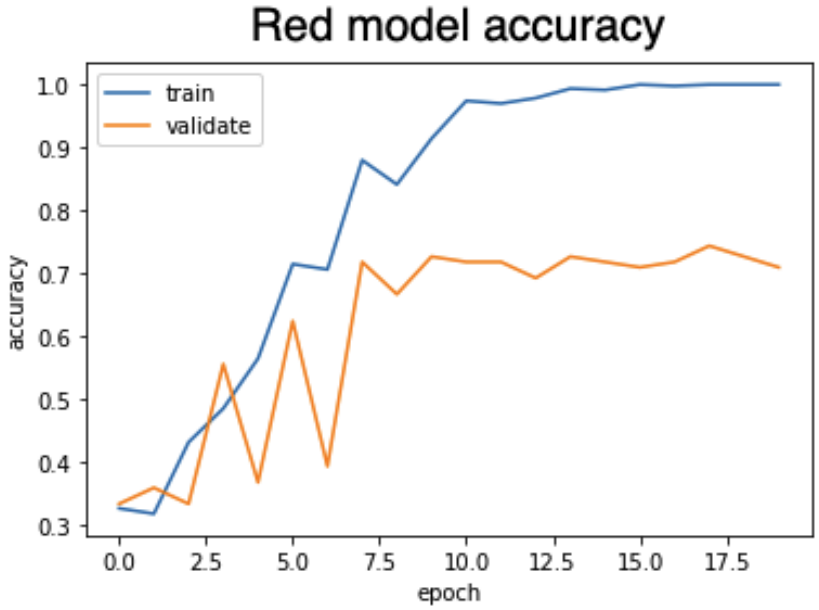


Figure 6.1: Red model: Training and validation accuracy from randomly selected fold during final evaluation.

6.3 Metrics and Confusion Matrices

As we can see in Table 6.2, the weighted ensemble of the models provides a significant increase in accuracy as compared to either of the models individually. The green model (coronal plane) produces the best accuracy. It is important to note that the standard deviation here is reliant on how each of the folds evaluates the models. The high standard deviation that occurs here is because the only accuracies each of the models can have are either 0.0, 0.33, 0.67, or 1.0. If a model is trained on more samples, each fold would be more representative of the true accuracy of the model. This is likely to cause the standard deviation to decrease – therefore, the standard deviation, in this case, should only be compared to other model validation techniques that are also evaluated based on the same test set size as done here. Furthermore, despite promising results, we can clearly tell that the MCI class is the hardest one to predict accurately.

Table 6.1: Final Model Evaluation Metrics

Metric	Score	Standard Deviation
Accuracy	0.8271604938271605	0.2102393378509432
Red Accuracy	0.7599451303155006	0.2409507093995478
Green Accuracy	0.7599451303155006	0.2409507093995478
Blue Accuracy	0.7709190672153634	0.24402363018217563
Log Loss	0.6830885245966813	0.1442741702639510

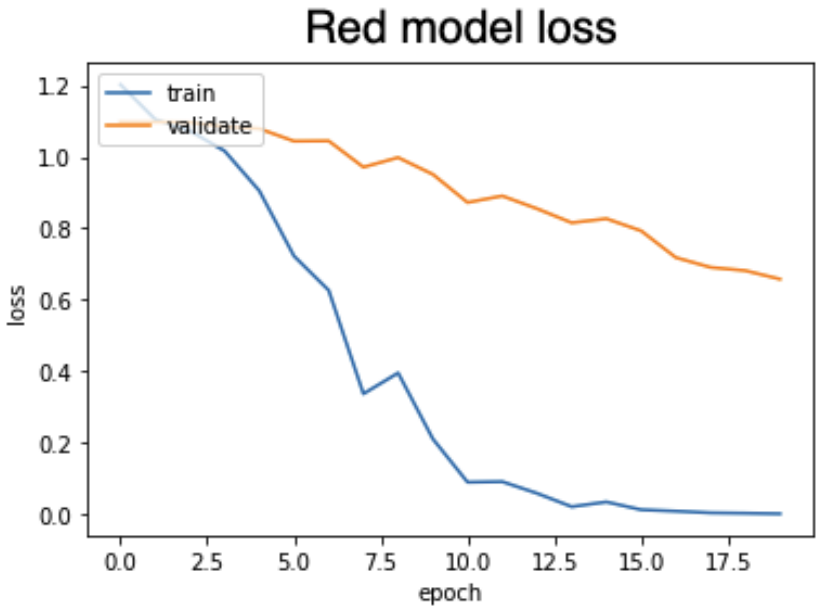


Figure 6.2: Red model: Training and validation loss from randomly selected fold during final evaluation.

Figure 6.7 is a confusion matrix which visualises the prediction of our model. It makes it easier and more intuitive to see where improvements can be made, and where effort should be focused in future models. The confusion matrix shows that the main false positives end up in the neighboring classes. This makes sense as both the matrix, the softmax and the logical progression of AD has MCI being the middle-stage. The model manages to clearly differentiate between AD and NC with only one AD predicted as NC, and vice versa. Furthermore, the model is slightly more likely to wrongfully predict NC as MCI, than AD as MCI. Whether this is because of the similarity of the stages, or other factors, are unknown.

To be able to generate recall, precision, and F1-scores from this data, we will need to find the true-positives, true-negatives, false-positives, and false-negatives for each of the classes in Figure 6.7. Whilst we can derive these from Figure 6.7, it is more helpful to create a 2×2 confusion matrix for each of the classes, and then calculate their respective metrics from there.

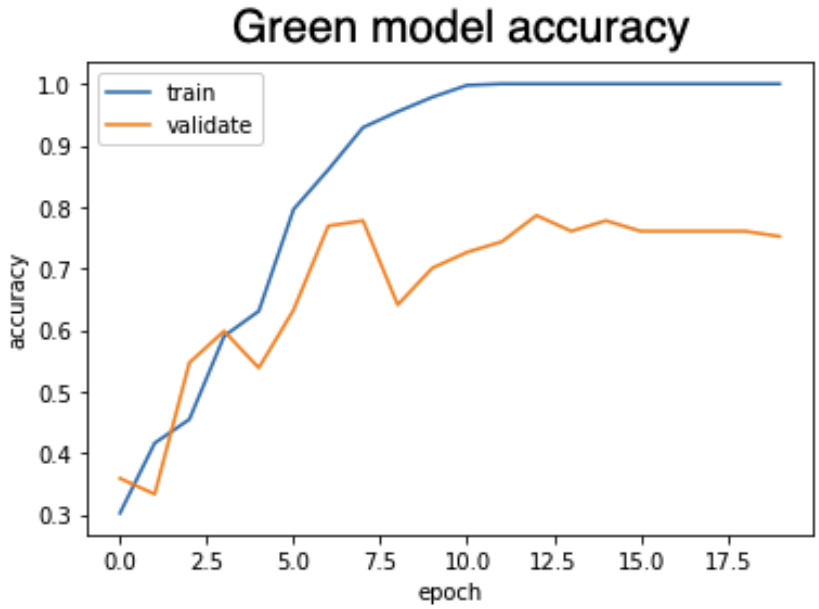


Figure 6.3: Green model: Training and validation loss from randomly selected fold during final evaluation.

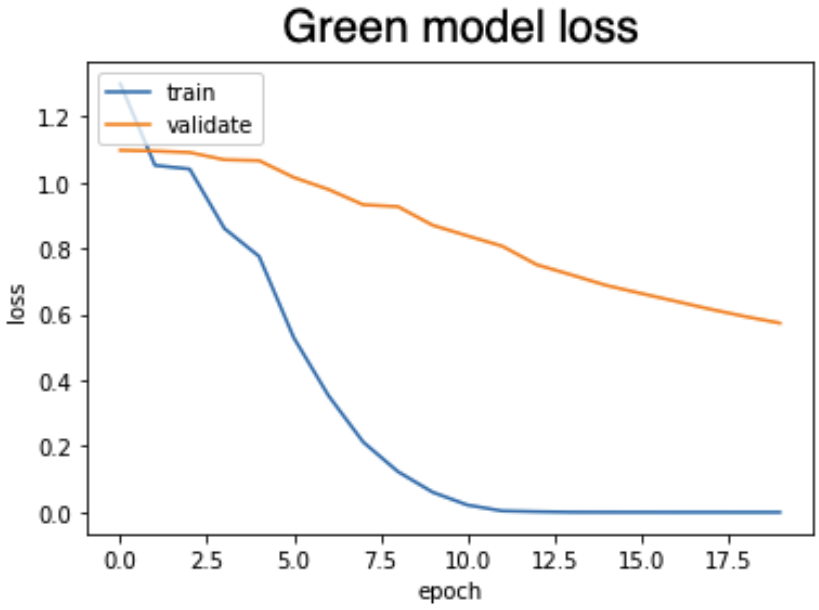


Figure 6.4: Green model: Training and validation loss from randomly selected fold during final evaluation.

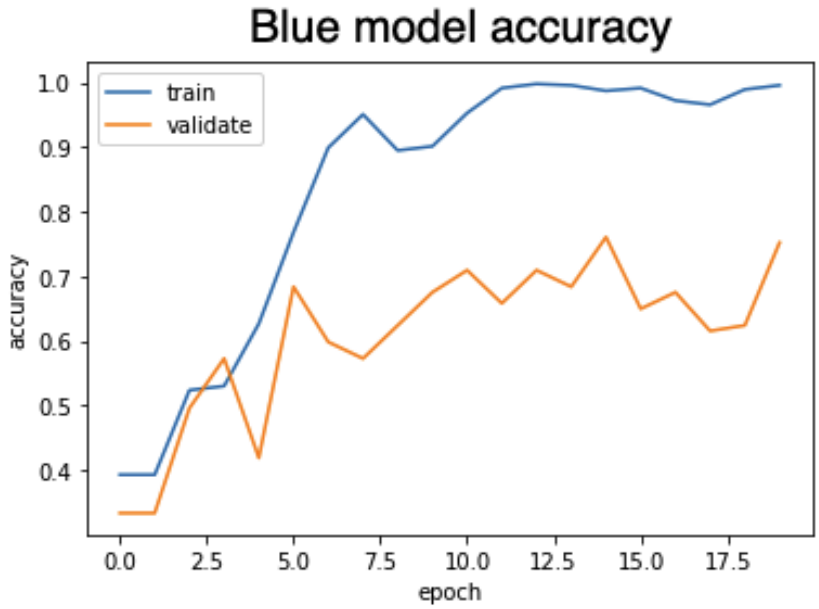


Figure 6.5: Blue model: Training and validation loss from randomly selected fold during final evaluation.

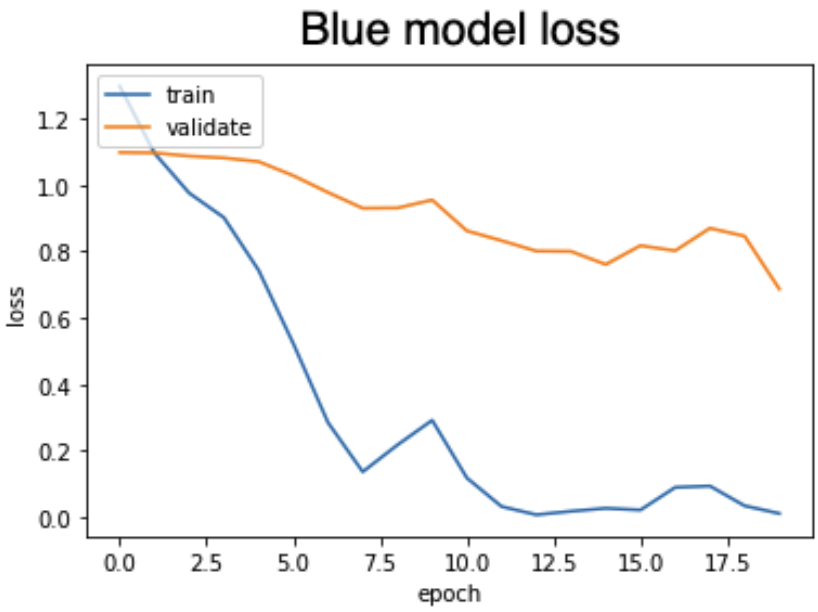


Figure 6.6: Blue model: Training and validation loss from randomly selected fold during final evaluation.

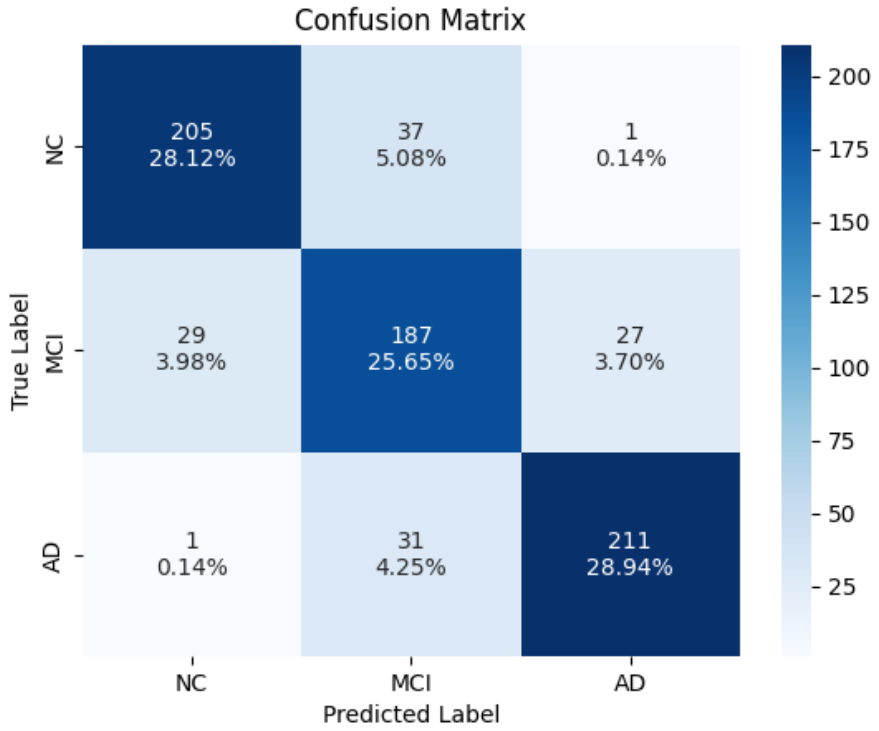


Figure 6.7: Result Confusion Matrix

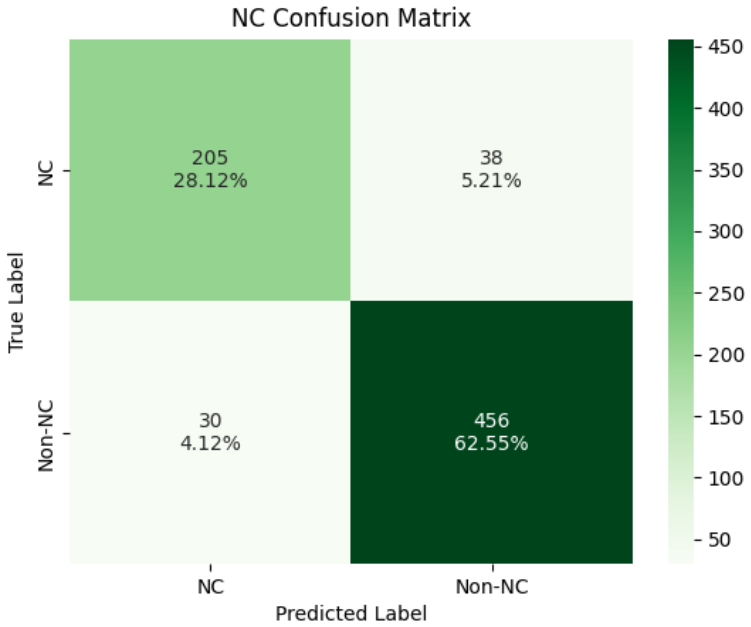


Figure 6.8: NC Confusion Matrices

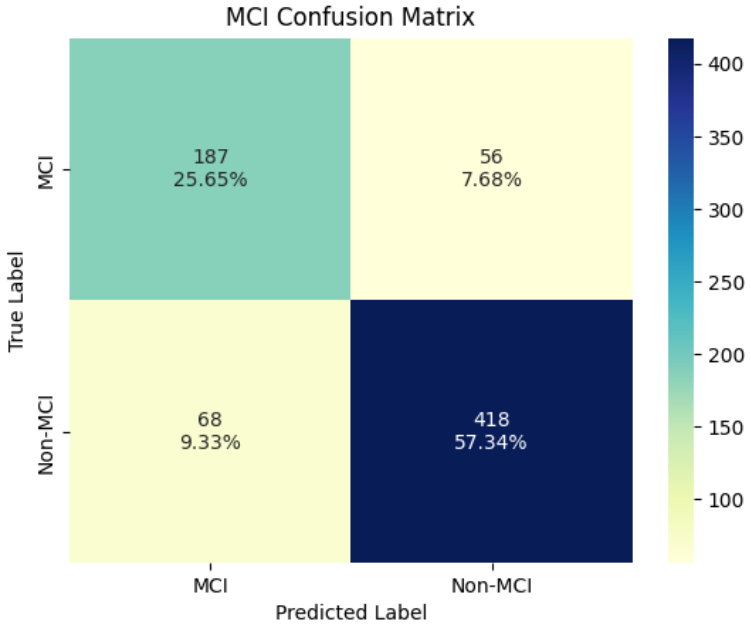


Figure 6.9: MCI Confusion Matrices

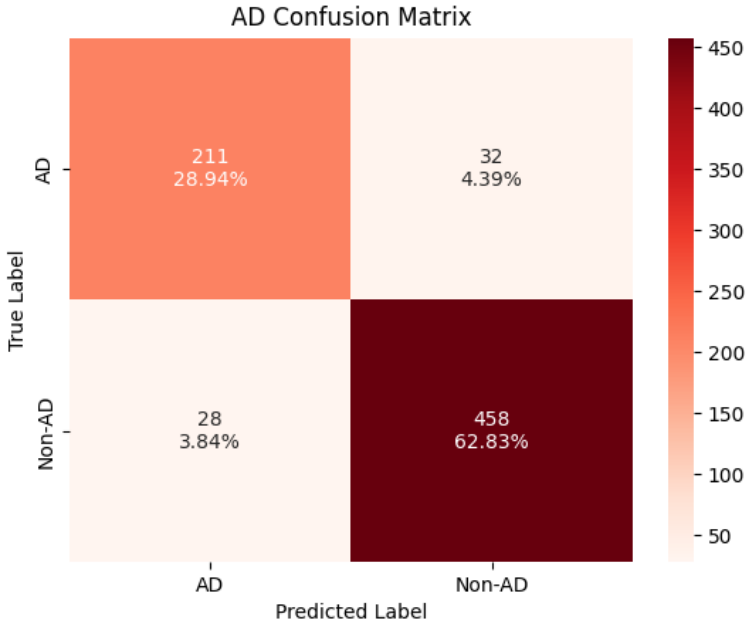


Figure 6.10: AD Confusion Matrices

Table 6.2: Class Specific Evaluation Metrics

Class	Precision	Recall	F1-Score
NC	0.87234042553	0.8436213991	0.85774058576
MCI	0.73333333333	0.7695473251	0.75100401606
AD	0.88284518828	0.8683127572	0.87551867219

Similar to the 3×3 matrix presented in Figure 6.7, we can tell from the matrices Figure 6.8, Figure 6.9, and Figure 6.10 that the MCI class is the hardest one to classify. The precision, recall and F1-score from NC and AD are all good, and they all improve on the previous years metrics.

6.4 Discussion

As indicated by the literature (see section 3.1), the MCI class can be the most challenging, whilst arguably also the most important area to predict for a model. In this area the model achieved significant results, as it got a MCI recall of 76.9%, which surpass the 72.4% Lackey [15] obtained in 2021. Recall is in this case equivalent to recall (see Table 2.1). The NC class got the the same as results as Lackey [15], with a retrieval of 205/243 correct samples. Whilst for AD the recall was slightly better with this model acquiring 86.8% recall, as opposed to the 85.1% obtained by Lackey [15]. The results here are only compared to Lackey [15], as she had the best results compared to that of Guo [70], Bao [71] and Chao [72].

The final model, including the AD, MCI and NC classes, reached an overall accuracy of 82.7% which is an improvement on the 80.7% accuracy achieved by Lackey [15]. With the best individual model obtaining an accuracy of 77.9% (green model, coronal plane). The weighted ensemble model produces a superior overall accuracy compared to the best individual model, proving the efficiency of the approach. An interesting observation of our result is that whilst our overall accuracy has improved, the individual models are on all instances slightly worse than that of Lackey [15]. The difference in accuracy is small, and it is less than 1% in difference for all the models.

It was observed that the model was able to more accurately predict AD than NC (see Figure 6.7). This could be because the NC class has more overlap with the MCI class than the AD class has to MCI. However, because of the limited dataset, this could also have been caused by either a lack of generalisation or a slightly biased dataset.

The noticeable difference in activation function performance is also interesting. Finding that two very similar activation functions (SeLU and LeakyReLU, see Figure 5.8) were on opposite sides in terms of performance was surprising. The reason for this is not certain, however, activation functions relying on non-linear attributes have in more recent years been replaced by more linear activation functions such as the LeakyReLU [73].

An attempt to create a 3D architecture was also made, however due to the computational difficulties it introduced, the 2.5D CNN was chosen. An explanation of work done towards

the 3D CNN can be found in Appendix G.

6.5 Reflection

Looking back at the project, staying consistent and organised was never an issue. Fumie's weekly progression meetings made this easy, as it was a constant deadline where one wanted to show up with something.

The most used tools during this process was Python 3 for programming, the most used libraries were Tensorflow, Keras and Numpy and for execution Google Colab Pro was used. The extra computational power provided by Google Colab's GPU accelerator made it possible to could run multiple experiments a day, something which the M1 Mac 2020 was struggling with. Specifically, having a k-fold run in 2-hours, compared to 2 days, was very beneficial. The Python libraries functionality was essential, however there was alternatives. The main benefit of these libraries were their range of supporting information, both in terms of documentation and through support on Q&A forums such as Stack Overflow. It also helped that these were heavily used by previous students, thus examples of code were easy to come by. ImageJ was used for analysing the MRI scans. It made understanding and visualising segmentation and ROI extraction much more intuitive.

In terms of experimentation, figuring out that certain research pipelines would be out of the scope of this project early was important. Despite the value one can gain from researching 3D segments of the brain, the computational demands proved to be too much for Google Colab to handle with a reasonable ML architecture.

Being able to access previous years' code has also helped reduce the complexity of this year's project. Being able to run different models to see how they differ, and as well as comparing architecture and performance, gave a good indicator of where to start.

Chapter 7

Conclusion and Future Work

7.1 Summary

The objective of this project was to create an ML classifier that could classify the state of a brain into either AD, MCI, or NC. The project kept a specific focus on improving the accuracy within the MCI class because of the existence of treatments and medication to slow down the progression from MCI to AD. General performance was limited by a small dataset - something which is a common issue within medical models.

A weighted ensemble model was created by combining three models, each of which had been trained only on one of the input planes. The resulting model used five convolutional layers with pooling layers spread in-between together with the LeakyReLU activation function. This is followed by batch normalisation and flattening, before three hidden layers, and lastly, a dropout layer.

7.2 Conclusion

The model got an overall accuracy of 82.7% surpassing the 80.7% accuracy achieved by Lackey [15]. Furthermore, it obtained a 76.9% accuracy within the MCI class, which also surpassed that of previous years, the best one being the 72.4% accuracy obtained by Lackey [15]. The model performs worse than state-of-the-art methods (see Table 3.1). However, considering the limited training data this is to be expected. Furthermore, the used architecture had a big focus on reducing computational complexity because of the limitation in hardware. This comes at the cost of performance when we compare it to more complex networks (see Table 3.1). Despite the improvements within the MCI class, its performance is still the lowest of the classes. However, that is to be expected considering that the MCI class has more 3-way overlap than that of AD and NC. Ultimately, the model was still effective in learning the underlying patterns of the data necessary for accurate classification.

7.2.1 Evaluation of Objectives

1. Data Preprocessing

(a) The dataset created by Lackey [15] was successfully applied and showed great usability. Before the dataset was utilised, the dimensionality of the images was reduced to a size of 64×64 in accordance with Lackey's [15] approach. Furthermore, a good understanding of the preprocessing techniques and tools was established, both through research of previous work, but also through experimentation done regarding the previously mentioned 3D CNN.

2. CNN

   (a) A weighted-ensemble model was created from three separate models, each trained on 2D slices from separate anatomical planes. The model managed to achieve an overall accuracy of 82.7%, improving the 80.7% accuracy obtained by Lackey [15]. However, the model did still show signs of overfitting. On the other hand, this is likely to be inevitable because of the restricted dataset.

3. MCI Performance

   (a) A focus was put on the MCI class performance. The goal was reached as the final model got a 76.9% accuracy in the MCI class, which is 4.5% better than the 72.4% acquired by Lackey [15]. As seen in Figure 6.8 - Figure 6.10 and Table 6.2, the MCI class is still the worst-performing class. However, avoiding this is probably very hard as the MCI class has more neurological overlap with both AD and NC, than AD and NC have with each other.

4. Small Dataset

   (a) The small dataset originally contained 1070 MRI brain scans. Testing and previous work showed that avoiding class imbalance gave the model the best results, hence, the dataset used was further reduced to 729 samples (243 samples per class) [15]. This shows that despite the already limited dataset, preventing the model from overfitting towards a specific class is more important than having additional training samples.

Considering the state of these objectives, we can conclude that the overall goal of the project has been successful.

7.3 Future Work

The model introduced in this project increases overall and MCI class accuracy. The use of a weighted ensemble model proved that despite the close performance of the anatomical planes, some additional performance could still be extracted. Despite the increase in performance displayed by this model, there are still several aspects that can be investigated further. A core issue with this model is the restricted dataset used to train it. Lackey [15] found preprocessing steps to not impact the performance of the model, however, only a limited number of data augmentation techniques were applied. Data augmentation methods

such as cropping, flipping, and deformation should be explored further before a conclusion on its effect can be drawn.

To further increase performance it could be worth doing further analysis on the input ROIs. Currently, only the seven regions outlined by Dai [14] are used. However, while there is some evidence that this combination of ROIs generates a good representation of the atrophy of AD, it is not necessarily the best combination of ROIs in regard to the model's performance. Hence, deeper research in the area could prove very beneficial; not only to improve the performance of this model, but for other AD classification architectures that depend on MRI scans. A potential approach could be training separate models on individual ROIs to find the best performing regions, both in terms of overall performance and MCI performance. The results of these experiments could then be combined to create a better combination of predictive structures.

Another optimisation worth considering is the use of transfer learning. Transfer learning is where you initialise the weights of a new model, with the weights of a pre-trained model from a similar problem. For this project, that would include researching other areas where 2D MRI slices have been used for classification and testing the results of initialising our model's weights with the final weights of researched models. Pre-trained models have the chance of having experienced more training data. Hence, certain brain patterns could potentially be learnt better than what is possible with the limited dataset used for this project.

Lastly, further investigations into different types of 3D architectures could prove beneficial as shown in previous studies. A potential 3D architecture can seen in Appendix G. However, the important consideration here is the availability of computational power. Therefore, this area should only be explored if reasonable arguments for the computational feasibility of the architectures are introduced.

Bibliography

[1] Z. Breijyeh and R. Karaman, "Comprehensive review on alzheimer's disease: Causes and treatment," *Molecules*, vol. 25, no. 24, p. 5789, 2020.

[2] S. Basaia, F. Agosta, L. Wagner, E. Canu, G. Magnani, R. Santangelo, and M. Filippi, "Automated classification of alzheimer's disease and mild cognitive impairment using a single mri and deep neural networks," *NeuroImage: Clinical*, vol. 21, p. 101645, 12 2018.

[3] M. Peixeiro. Step-by-step Guide to Building Your Own Neural Network From Scratch. [Online]. Available: https://towardsdatascience.com/step-by-step-guide-to-building-your-own-neural-network-from-scratch-df64b1c5ab6e

[4] K. Krzyk. Coding Deep Learning for Beginners — Linear Regression (Part 3): Training with Gradient Descent. [Online]. Available: https://towardsdatascience.com/coding-deep-learning-for-beginners-linear-regression-gradient-descent-fcd5e0fc077d

[5] C. Goyal. 20 Questions to Test your Skills on CNN (Convolutional Neural Networks). [Online]. Available: https://www.analyticsvidhya.com/blog/2021/05/20-questions-to-test-your-skills-on-cnn-convolutional-neural-networks/

[6] K. E. Swapna. Convolutional Neural Network — Deep Learning. [Online]. Available: https://developersbreach.com/convolution-neural-network-deep-learning/

[7] S. Albawi, T. A. Mohammed, and S. Al-Zawi, "Understanding of a convolutional neural network," in *2017 International Conference on Engineering and Technology (ICET)*, 2017, pp. 1–6.

[8] S. Shrivastav. Tutorial — How to visualize Feature Maps directly from CNN layers. [Online]. Available: https://www.analyticsvidhya.com/blog/2020/11/tutorial-how-to-visualize-feature-maps-directly-from-cnn-layers/

[9] D. A. Nguyen. Hyperparameters. [Online]. Available: https://eng.ftech.ai/?p=642

[10] E. Anello. K-Fold Cross Validation for Machine Learning Models. [Online]. Available: https://pub.towardsai.net/k-fold-cross-validation-for-machine-learning-models-918f6ccfd6d

[11] FSL. Templates and Atlases included with FSL. [Online]. Available: https://fsl.fmrib.ox.ac.uk/fsl/fslwiki/Atlases

[12] W. Lin, T. Tong, Q. Gao, D. Guo, X. Du, Y. Yang, G. Guo, M. Xiao, M. Du, X. Qu, and T. A. D. N. I. , "Convolutional neural networks-based mri image analysis for the alzheimer's disease prediction from mild cognitive impairment," *Frontiers in Neuroscience*, vol. 12, 2018. [Online]. Available: https://www.frontiersin.org/article/10.3389/fnins.2018.00777

[13] Iglesias, Juan Eugenio. ROBEX. [Online]. Available: https://sites.google.com/site/jeiglesias/ROBEX

[14] S. Dai, "A new method for early diagnosis of alzheimer's disease by convolutional neural network, based on 2.5d slices extraction," *University of Manchester Doctor of Philosophy, Faculy of Science and Engineering*, 2019.

[15] A. Lackey, "Classification of Medical Images into AD, MCI and Healthy Brain," *University of Manchester Third Year Project*, 2021.

[16] C. Hansen. Activation Functions Explained - GELU, SELU, ELU, ReLU and more. [Online]. Available: https://mlfromscratch.com/activation-functions-explained/#/

[17] G. L. Wenk, "Neuropathologic changes in alzheimer's disease," *Journal of Clinical Psychiatry*, vol. 64, pp. 7–10, 2003.

[18] J. Weller and A. Budson, "Current understanding of alzheimer's disease diagnosis and treatment," *F1000Research*, vol. 7, 2018.

[19] B. Reisberg and E. H. Franssen, "Clinical stages of alzheimer's disease," *An atlas of Alzheimer's disease. New York, London: The Parthenon Publishing Group*, pp. 11–29, 1999.

[20] T. D. Vu, H.-J. Yang, V. Q. Nguyen, A.-R. Oh, and M.-S. Kim, "Multimodal learning using convolution neural network and sparse autoencoder," in *2017 IEEE International Conference on Big Data and Smart Computing (BigComp)*, 2017, pp. 309–312.

[21] A. Payan and G. Montana, "Predicting alzheimer's disease: a neuroimaging study with 3d convolutional neural networks," *ICPRAM 2015 - 4th International Conference on Pattern Recognition Applications and Methods, Proceedings*, vol. 2, 02 2015.

[22] A. Valliani and A. Soni, "Deep residual nets for improved alzheimer's diagnosis," in *Proceedings of the 8th ACM international conference on bioinformatics, computational biology, and health informatics*, 2017, pp. 615–615.

[23] C. Aggarwal, *Neural Networks and Deep Learning: A Textbook*. Springer International Publishing, 2018. [Online]. Available: https://books.google.co.uk/books?id=achqDwAAQBAJ

[24] Y. Gupta, R. K. Lama, G.-R. Kwon, M. W. Weiner, P. Aisen, M. Weiner, R. Petersen, C. R. Jack Jr, W. Jagust, J. Q. Trojanowki *et al.*, "Prediction and classification of alzheimer's disease based on combined features from apolipoprotein-e genotype, cerebrospinal fluid, mr, and fdg-pet imaging biomarkers," *Frontiers in computational neuroscience*, p. 72, 2019.

[25] R. Cemental. How Common Is Alzheimer's Disease? [Online]. Available: https://www.caringseniorservice.com/blog/alzheimers-statistics

[26] A. Association.

[27] M. Crous-Bou, C. Minguillón, N. Gramunt, and J. L. Molinuevo, "Alzheimer's disease prevention: from risk factors to early intervention," *Alzheimer's research & therapy*, vol. 9, no. 1, pp. 1–9, 2017.

[28] M. J. Prince, A. Wimo, M. M. Guerchet, G. C. Ali, Y.-T. Wu, and M. Prina, "World alzheimer report 2015-the global impact of dementia: An analysis of prevalence, incidence, cost and trends," 2015.

[29] National Insitute of Aging. What Causes Alzheimer's Disease? [Online]. Available: https://www.nia.nih.gov/health/what-causes-alzheimers-disease

[30] S. Wang, P. N. Mims, R. J. Roman, and F. Fan, "Is beta-amyloid accumulation a cause or consequence of alzheimer's disease?" *Journal of Alzheimer's parkinsonism & dementia*, vol. 1, no. 2, 2016.

[31] Alzheimer's Association. Stages of Alzheimer's. [Online]. Available: https://www.alz.org/alzheimers-dementia/stages

[32] M. W. Bondi, E. C. Edmonds, A. J. Jak, L. R. Clark, L. Delano-Wood, C. R. McDonald, D. A. Nation, D. J. Libon, R. Au, D. Galasko *et al.*, "Neuropsychological criteria for mild cognitive impairment improves diagnostic precision, biomarker associations, and progression rates," *Journal of Alzheimer's Disease*, vol. 42, no. 1, pp. 275–289, 2014.

[33] R. C. Petersen, G. E. Smith, S. C. Waring, R. J. Ivnik, E. Kokmen, and E. G. Tangelos, "Aging, memory, and mild cognitive impairment," *International psychogeriatrics*, vol. 9, no. S1, pp. 65–69, 1997.

[34] S. A. Mofrad, A. J. Lundervold, A. Vik, and A. S. Lundervold, "Cognitive and mri trajectories for prediction of alzheimer's disease," *Scientific Reports*, vol. 11, no. 1, pp. 1–10, 2021.

[35] Z. S. Khachaturian, "Diagnosis of alzheimer's disease," *Archives of neurology*, vol. 42, no. 11, pp. 1097–1105, 1985.

[36] U.S. National Institute on Aging. How Is Alzheimer's Disease Diagnosed? [Online]. Available: https://www.nia.nih.gov/health/how-alzheimers-disease-diagnosed#:~:text=Perform%20brain%20scans%2C%20such%20as,other%20possible%20causes%20for%20symptoms.

[37] Mayo Clinic. Alzheimer's: Drugs help manage symptoms. [Online]. Available: https://www.mayoclinic.org/diseases-conditions/alzheimers-disease/in-depth/alzheimers/art-20048103#:~:text=Alzheimer's%20still%20has%20no%20cure,symptoms%20and%20prolong%20your%20independence.

[38] P. J. Kumar and M. L. Clark, *Kumar Clark's clinical medicine*, 8th ed., ser. Kumar, Kumar and Clark's Clinical Medicine. Elsevier/Saunders, 2012.

[39] T. Mitchell, *Machine Learning*, ser. McGraw-Hill International Editions. McGraw-Hill, 1997. [Online]. Available: https://books.google.co.uk/books?id=EoYBngEACAAJ

[40] E. Alpaydin, *Introduction to Machine Learning*, ser. Adaptive Computation and Machine Learning series. MIT Press, 2014. [Online]. Available: https://books.google.co.uk/books?id=NP5bBAAAQBAJ

[41] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, ser. Always learning. Pearson, 2016. [Online]. Available: https://books.google.co.uk/books?id=XS9CjwEACAAJ

[42] T. C. Redman. If Your Data Is Bad, Your Machine Learning Tools Are Useless. [Online]. Available: https://hbr.org/2018/04/if-your-data-is-bad-your-machine-learning-tools-are-useless

[43] R. Kenett and T. Redman, *The Real Work of Data Science: Turning data into information, better decisions, and stronger organizations*. Wiley, 2019. [Online]. Available: https://books.google.co.uk/books?id=jXpbvwEACAAJ

[44] D. Brain and G. Webb, "On the effect of data set size on bias and variance in classification learning," *Proceedings of the Fourth Australian Knowledge Acquisition Workshop*, 06 2000.

[45] R. L. Melvin. Sample Size in Machine Learning and Artificial Intelligence. [Online]. Available: https://sites.uab.edu/periop-datascience/2021/06/28/sample-size-in-machine-learning-and-artificial-intelligence/

[46] G. V. Trunk, "A problem of dimensionality: A simple example," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-1, no. 3, pp. 306–307, 1979.

[47] S. C. Wong, A. Gatt, V. Stamatescu, and M. D. McDonnell, "Understanding data augmentation for classification: When to warp?" in *2016 International Conference on Digital Image Computing: Techniques and Applications (DICTA)*, 2016, pp. 1–6.

[48] A. Takimoglu. What is Data Augmentation? Techniques Examples in 2022. [Online]. Available: https://research.aimultiple.com/data-augmentation/

[49] M. Bari Antor, A. Jamil, M. Mamtaz, M. Monirujjaman Khan, S. Aljahdali, M. Kaur, P. Singh, and M. Masud, "A comparative analysis of machine learning algorithms to predict alzheimer's disease," *Journal of Healthcare Engineering*, vol. 2021, 2021.

[50] L. V. Fulton, D. Dolezel, J. Harrop, Y. Yan, and C. P. Fulton, "Classification of alzheimer's disease with and without imagery using gradient boosted machines and resnet-50," *Brain sciences*, vol. 9, no. 9, p. 212, 2019.

[51] J. Hopfield, "Neural networks and physical systems with emergent collective computational abilities," *Proceedings of the National Academy of Sciences of the United States of America*, vol. 79, pp. 2554–8, 05 1982.

[52] S.-C. Wang, *Artificial Neural Network*. Boston, MA: Springer US, 2003, pp. 81–100. [Online]. Available: https://doi.org/10.1007/978-1-4615-0377-4_5

[53] W. S. McCulloch and W. Pitts, "A logical calculus of the ideas immanent in nervous activity," *The bulletin of mathematical biophysics*, vol. 5, no. 4, pp. 115–133, 1943.

[54] A. Géron, *Neural Networks and Deep Learning*. O'Reilly, 2018. [Online]. Available: https://books.google.co.uk/books?id=5pm6tQEACAAJ

[55] F. Chollet, *Deep Learning with Python*. Manning, 2017. [Online]. Available: https://books.google.co.uk/books?id=wzozEAAAQBAJ

[56] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.

[57] J. Brownlee. Gentle Introduction to the Adam Optimization Algorithm for Deep Learning. [Online]. Available: https://machinelearningmastery.com/adam-optimization-algorithm-for-deep-learning/#:~:text=Adam%20is%20a%20replacement%20optimization,sparse%20gradients%20on%20noisy%20problems.

[58] E. Tuba, N. Bačanin, I. Strumberger, and M. Tuba, *Convolutional Neural Networks Hyperparameters Tuning*. Cham: Springer International Publishing, 2021, pp. 65–84. [Online]. Available: https://doi.org/10.1007/978-3-030-72711-6_4

[59] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, ser. Adaptive Computation and Machine Learning series. MIT Press, 2016. [Online]. Available: https://books.google.co.uk/books?id=Np9SDQAAQBAJ

[60] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014. [Online]. Available: http://jmlr.org/papers/v15/srivastava14a.html

[61] S. Sharma, S. Sharma, and A. Athaiya, "Activation functions in neural networks," *towards data science*, vol. 6, no. 12, pp. 310–316, 2017.

[62] J. Brownlee. Understand the Impact of Learning Rate on Neural Network Performance. [Online]. Available: https://machinelearningmastery.com/understand-the-dynamics-of-learning-rate-on-deep-learning-neural-networks/

[63] M. Hossin and S. M.N, "A review on evaluation metrics for data classification evaluations," *International Journal of Data Mining Knowledge Management Process*, vol. 5, pp. 01–11, 03 2015.

[64] M. Grandini, E. Bagli, and G. Visani, "Metrics for multi-class classification: an overview," 08 2020.

[65] A. Dyakonov. Log Loss Function. [Online]. Available: https://dasha.ai/en-us/blog/log-loss-function

[66] P. Refaeilzadeh, L. Tang, and H. Liu, *Cross-Validation*. Boston, MA: Springer US, 2009, pp. 532–538. [Online]. Available: https://doi.org/10.1007/978-0-387-39940-9_565

[67] S. Cai, K. Huang, Y. Kang, Y. Jiang, K. Von Deneen, and L. Huang, "Potential biomarkers for distinguishing people with alzheimer's disease from cognitively intact elderly based on the rich-club hierarchical structure of white matter networks," *Neuroscience Research*, vol. 144, 08 2018.

[68] O. Sagi and L. Rokach, "Ensemble learning: A survey," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 8, no. 4, p. e1249, 2018.

[69] D. Jiménez, "Dynamically weighted ensemble neural networks for classification," in *1998 IEEE International Joint Conference on Neural Networks Proceedings. IEEE World Congress on Computational Intelligence (Cat. No. 98CH36227)*, vol. 1. IEEE, 1998, pp. 753–756.

[70] A. Guo, "A Classification Method Of Brain Images Into AD, MCI And NC," *University of Manchester Master of Computer Science*, 2020.

[71] C. Bao, "Optimisation Using Bayesian Optimisation For Medical Image Classification In Alzheimer's Disease, Mild Cognitive Impairment And Healthy Brain," *University of Manchester Doctor of Philosophy*, 2019.

[72] F. Chao, "Classification Of Medical Images Into Alzheimer's Disease, Mild Cognitive Impairment And Healthy Brain," *University of Manchester Master Of Research In Infomatics*, 2018.

[73] G. E. Dahl, T. N. Sainath, and G. E. Hinton, "Improving deep neural networks for lvcsr using rectified linear units and dropout," in *2013 IEEE International Conference on Acoustics, Speech and Signal Processing*, 2013, pp. 8609–8613.

[74] Alzheimer's Society, "Drug treatments for alzheimer's disease factsheet 407lp," 2014.

[75] M. Smith. NMDA Receptor Antagonists and Alzheimer's. [Online]. Available: https://www.webmd.com/alzheimers/guide/nmda-receptor-antagonists#:~:text=NMDA%20(short%20for%20N%2Dmethyl,drugs%20may%20slow%20it%20down.

Appendix A

Ethics

The data used for this project originate from the ADNI database. "The Alzheimer's Disease Neuroimaging Initiative (ADNI) is a longitudinal multicenter study designed to develop clinical, imaging, genetic, and biochemical biomarkers for the early detection and tracking of Alzheimer's disease (AD). Since its launch more than a decade ago, the landmark public-private partnership has made major contributions to AD research, enabling the sharing of data between researchers around the world."

The data does not include any personal identifiers, nor any confidential information. Hence, none of the MRI scans can be traced back to the individual who participated. To further ensure ethical consistency, the Manchester University ethics decision tool [1] was used to evaluate the project. The conclusion was that there was no need for a formal ethical approval because of the nature of the work, and the anonymity of the data used.

[1] University of Manchester ethical approval tool, used to help to determine whether a project requires formal ethical approval or not: `https://www.manchester.ac.uk/research/environment/governance/ethics/approval/` (last accessed 18/04/2022)

Appendix B

Treatments

AD treatments as outlined by Kumar and Clark [38]:

- General Measures

  Some evidence indicate that engaging in cognitively demanding activities in later stages of life may help to protect against, or delay the onset of dementia. Other reports show that a high-dose B vitamins may slow down conversion from MCI to AD.

- Cognitive Enhancers

  Cognitive enhancers have a modest symptomatic benefit in AD, equal to an increase on the MMSE of 1 to 2 points. However, cognitive enhancers do not affect the disease itself, as they neither prevent nor slow down the progression. There is some dispute around these drugs in the treatment pipeline. They are expensive, however they also help prolong the patient's independence.

- Cholinesterase inhibitors

  Medication such as donepezil, rivastigmine and galantamine are used to increase brain acetylcholine levels inhibiting the brain's acetylcholinesterase. This leads to increased communication between nerve cells, which may temporarily alleviate or stabilise some symptoms of AD [74]. All three cholinesterase inhibitors are similar in the way they work, however one might suit a certain individual better. This is due to the different side effects experienced by the different drugs. However, none of these drugs will slow down the progression of AD [74].

- Memantine

  Memantine is an N-methyl-D-aspartate (NDMA) receptor antagonist. NDMA is a class of drugs that may help in the treatment AD by decreasing unusual brain activity [75]. Memantine is used in cases where the patient has either moderate or severe AD, or where cholinesterase inhibitors are not tolerated.

There exist some evidence that suggests that a combination of Memantine and cholinesterase inhibitors work better than either used independently.

- Psychiatric and Behavioural Problems

  Depression is a common occurrence in dementia patients, and it may be difficult to distinguish it from other symptoms such as apathy and a decline in cognitive functionality. It is recommended that a trial of antidepressants is tried where depression is suspected. Distinguishing depressive psuedo-dementia from organic dementia can be hard, yet important. Behavioural disturbance and delusions can occur in the later stages of AD. However, antipsychotic medication should only be used as a last resort for patients with dementia, as it can significantly increases the risk of stroke in a patient.

- Drugs in Development

  Disease modifying therapies that either halt, or slow down progression in early stage AD is something that is of great need. Some treatments in development include inhibitors of secretase enzymes that process Amyloid Precursor Protein (APP) into Aβ fragments and anti-amyloid therapies such as monoconal antibodies directed against Aβ.

Appendix C

Initial Model Architecture

Model Code:

```
# Model
def model():
    my_model = Sequential([
        Conv2D(32, kernel_size=3, input_shape=(64,64,1),
            data_format='channels_last',
            padding='same'),
        LeakyReLU(),
        Conv2D(64, kernel_size=3, padding='same'),
        LeakyReLU(),
        MaxPooling2D(pool_size=3),
        Conv2D(256, kernel_size=3, padding='same'),
        LeakyReLU(),
        MaxPooling2D(pool_size=3),
        Conv2D(256, kernel_size=3, padding='same'),
        LeakyReLU(),
        BatchNormalization(),
        MaxPooling2D(pool_size=2),

        Flatten(),
        Dense(410),
        LeakyReLU(),
        Dense(410),
        LeakyReLU(),
        Dense(410),
        LeakyReLU(),
        Dropout(0.25),
        Dense(3, activation='softmax')
    ])
```

```
    my_model.compile(
        optimizer=optimizers.Adam(learning_rate=0.00005),
        loss="categorical_crossentropy",
        metrics=["accuracy"]
    )

    return my_model
```

Model Summary:

Layer (type)	Output Shape	Param #
conv2d_2410 (Conv2D)	(None, 64, 64, 32)	320
activation_3775 (Activation)	(None, 64, 64, 32)	0
conv2d_2411 (Conv2D)	(None, 64, 64, 64)	18496
activation_3776 (Activation)	(None, 64, 64, 64)	0
max_pooling2d_1365 (MaxPooling2D)	(None, 21, 21, 64)	0
conv2d_2412 (Conv2D)	(None, 21, 21, 256)	147712
activation_3777 (Activation)	(None, 21, 21, 256)	0
max_pooling2d_1366 (MaxPooling2D)	(None, 7, 7, 256)	0
conv2d_2413 (Conv2D)	(None, 7, 7, 256)	590080
activation_3778 (Activation)	(None, 7, 7, 256)	0
batch_normalization_455 (BatchNormalization)	(None, 7, 7, 256)	1024
max_pooling2d_1367 (MaxPooling2D)	(None, 3, 3, 256)	0
flatten_455 (Flatten)	(None, 2304)	0

```
dense_1820 (Dense)           (None, 400)               922000

activation_3779 (Activation  (None, 400)               0
)

dense_1821 (Dense)           (None, 400)               160400

activation_3780 (Activation  (None, 400)               0
)

dense_1822 (Dense)           (None, 400)               160400

activation_3781 (Activation  (None, 400)               0
)

dropout_455 (Dropout)        (None, 400)               0

dense_1823 (Dense)           (None, 3)                 1203

=================================================================
Total params: 2,001,635
Trainable params: 2,001,123
Non-trainable params: 512
_________________________________________________________________
```

Appendix D

Final Model Architecture

Model Code:

```
# Model
def model():
    my_model = Sequential([
        Conv2D(32, kernel_size=3, input_shape=(64,64,1),
            data_format='channels_last',
            padding='same'),
        LeakyReLU(),
        Conv2D(64, kernel_size=3, padding='same'),
        LeakyReLU(),
        MaxPooling2D(pool_size=3),
        Conv2D(128, kernel_size=3, padding='same'),
        LeakyReLU(),
        Conv2D(256, kernel_size=3, padding='same'),
        LeakyReLU(),
        MaxPooling2D(pool_size=3),
        Conv2D(256, kernel_size=3, padding='same'),
        LeakyReLU(),
        BatchNormalization(),
        MaxPooling2D(pool_size=2),

        Flatten(),
        Dense(460),
        LeakyReLU(),
        Dense(460),
        LeakyReLU(),
        Dense(460),
        LeakyReLU(),
        Dropout(0.20),
        Dense(3, activation='softmax')
    ])
```

```
    my_model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=0.0005),
        loss="categorical_crossentropy",
        metrics=["accuracy"]
    )

    return my_model
```

Model Summary:

Layer (type)	Output Shape	Param #
conv2d_5 (Conv2D)	(None, 64, 64, 32)	320
leaky_re_lu_8 (LeakyReLU)	(None, 64, 64, 32)	0
conv2d_6 (Conv2D)	(None, 64, 64, 64)	18496
leaky_re_lu_9 (LeakyReLU)	(None, 64, 64, 64)	0
max_pooling2d_3 (MaxPooling 2D)	(None, 21, 21, 64)	0
conv2d_7 (Conv2D)	(None, 21, 21, 128)	73856
leaky_re_lu_10 (LeakyReLU)	(None, 21, 21, 128)	0
conv2d_8 (Conv2D)	(None, 21, 21, 256)	295168
leaky_re_lu_11 (LeakyReLU)	(None, 21, 21, 256)	0
max_pooling2d_4 (MaxPooling 2D)	(None, 7, 7, 256)	0
conv2d_9 (Conv2D)	(None, 7, 7, 256)	590080
leaky_re_lu_12 (LeakyReLU)	(None, 7, 7, 256)	0
batch_normalization_1 (Batc hNormalization)	(None, 7, 7, 256)	1024
max_pooling2d_5 (MaxPooling 2D)	(None, 3, 3, 256)	0

```
flatten_1 (Flatten)          (None, 2304)              0
dense_4 (Dense)              (None, 460)               1060300
leaky_re_lu_13 (LeakyReLU)   (None, 460)               0
dense_5 (Dense)              (None, 460)               212060
leaky_re_lu_14 (LeakyReLU)   (None, 460)               0
dense_6 (Dense)              (None, 460)               212060
leaky_re_lu_15 (LeakyReLU)   (None, 460)               0
dropout_1 (Dropout)          (None, 460)               0
dense_7 (Dense)              (None, 3)                 1383
=================================================================
Total params: 2,464,747
Trainable params: 2,464,235
Non-trainable params: 512
_________________________________________________________________
```

Appendix E

Weighted ensemble script

```
# This script need to be run after the model training as it
# requires the average validation accuracy of each model,
# and all the predictions made (yhats)
combined_accuracy = []

# get mean validation accuracy from each of the classes
r = np.mean(combined_red_val_acc)
g = np.mean(combined_green_val_acc)
b = np.mean(combined_blue_val_acc)

# Iterate through all predictions and weigh
# them using the validation accuracy
k = 1
score = 0
length = 0
for t in prediction_tuples:
  yhats = t[0]
  testy = t[1]

  # multiply each models prediction with
  # their respective validation accuracy
  yhats[0] = r**k*yhats[0]
  yhats[1] = g**k*yhats[1]
  yhats[2] = b**k*yhats[2]

  yhats = np.array(yhats)

  # sum across ensemble members
  summed = np.sum(yhats, axis=0)

  # argmax across classes to get the
  # final ensemble model prediction
```

```
    result = np.argmax(summed, axis=1)

    combined_accuracy.append(accuracy)


print("------------------------------------------------------------")
print(f" Mean Ensemble Score = {sum(combined_accuracy)/len(combined_accuracy)}")
```

Appendix F

GPU and CPU specs

```
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2020 NVIDIA Corporation
Built on Mon_Oct_12_20:09:46_PDT_2020
Cuda compilation tools, release 11.1, V11.1.105
Build cuda_11.1.TC455_06.29190527_0
Thu Feb 10 18:54:31 2022
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 460.32.03    Driver Version: 460.32.03    CUDA Version: 11.2     |
|-------------------------------+----------------------+----------------------+
| GPU  Name        Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M. |
|                               |                      |               MIG M. |
|===============================+======================+======================|
|   0  Tesla K80           Off  | 00000000:00:04.0 Off |                    0 |
| N/A   57C    P8    33W / 149W |      0MiB / 11441MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------+
| Processes:                                                                  |
|  GPU   GI   CI        PID   Type   Process name                  GPU Memory |
|        ID   ID                                                   Usage      |
|=============================================================================|
|  No running processes found                                                 |
+-----------------------------------------------------------------------------+
Model name:          Intel(R) Xeon(R) CPU @ 2.30GHz
```

Appendix G

Proposed 3D CNN

One of the initial goals with this project was to either create or improve on an existing neural network. Multiple studies showed great results using autoencoders and 3D MRI brain scans as seen in Table 3.1. Below is a proposed network where development was halted due to its high computational requirements.

G.1 3D CNN

The network introduced by Vu et al. [20] showed the effect of using both MRI scans and PET scans together. However, the dimensionality of the MRI scans caused less than 10% of the original features in the MRI scan to be kept. Resizing the MRI scan can cause that details are lost in the process. The proposed network gets around this by including important ROIs with the smaller MRI scan of the full brain. This network would therefore get the benefit of the entire brain scan, and the details of smaller important ROIs. This network would also apply a sparse autoencoder to pre-train the convolutional layers. The architecture with one ROI input can be seen in Figure G.1 (dimensions of the different layers are not specified as it would depend on the input data).

The model input would be similar to that seen in Figure G.2. The full brain (left) would be scaled down to a manageable size similar to that done by Vu et al. [20], and used as one input. The hippocampus (right) is then extracted and gets a new bounding box of smaller dimensions. This would be used as the second input - there is also the option to expand the architecture and add more ROIs. The script for giving a ROI a new bounding box is found in Appendix H.

Due to the need for a lot of computational power, the development of a model based on this architecture was stopped.

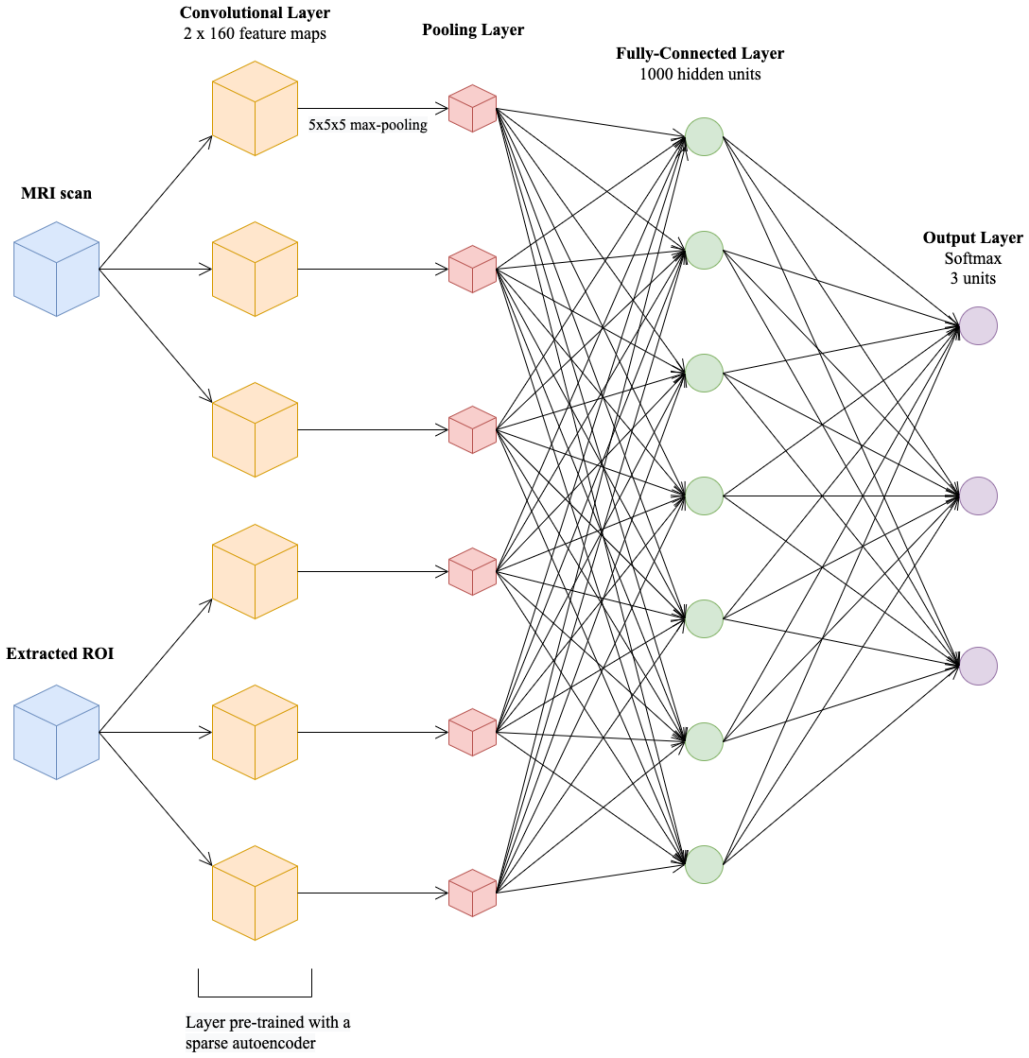


Figure G.1: Proposed 3D CNN

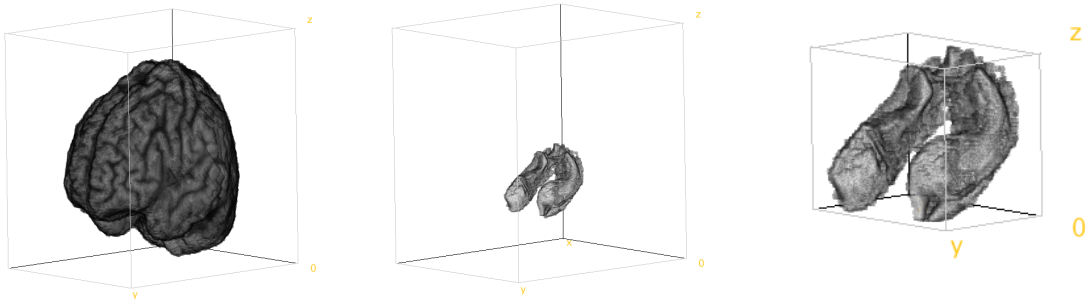


Figure G.2: Hippocampus extraction and bounding box adjustment

Appendix H

Bounding Box Minimisation

```
import nibabel as nib
import numpy as np
import matplotlib.pyplot as plt
import os
import skimage.measure

# ## Minimising scan dimensions

IN_PATH = "OutputFiles/05_ROI/"
OUT_PATH = "OutputFiles/06_ResizedROI/"

# change tag to resize another set of images
TAG = "_" # e.g. "_5_HippocampusROI_"
TAG = "_5_AmygdalaROI_"
Z, Y, X = 182, 218, 182
locations = [np.array([[float(0)]*X for _ in range(Y)]) for _ in range(Z)]
data = []
count = 0
max_val = 0
for file in os.listdir(IN_PATH):
    if file.endswith(".nii.gz") and TAG in file:
        count += 1
        print(count)
        nii_img = nib.load(IN_PATH + file)
        nii_data = nii_img.get_fdata()
        locations += nii_data
        data.append(nii_data)
print(max_val)


unused = 0
for i in range(X):
```

```
    for j in range(Y):
        for k in range(Z):
            if locations[i][j][k] == 0:
                unused += 1

print("Possible minimize factor:")
print((X*Y*Z-unused)/(X*Y*Z))

# Find outmost voxels in all 6 directions

left_X = 0
right_X = X
left_Y = 0
right_Y = Y
left_Z = 0
right_Z = Z

for i in range(X):
    s1 = locations[i,:,:].flatten()
    contains = False
    for i in s1:
        if i > 0:
            contains = True
            break
    if not contains:
        left_X += 1
    else:
        break

for i in range(X):
    s1 = locations[-(i+1)].flatten()
    contains = False
    for i in s1:
        if i > 0:
            contains = True
            break
    if not contains:
        right_X -= 1
    else:
        break

for i in range(Y):
    s1 = locations[:,i,:].flatten()
    contains = False
    for i in s1:
```

```
            if i > 0:
                contains = True
                break
        if not contains:
            left_Y += 1
        else:
            break

    for i in range(Y):
        s1 = locations[:,-(i+1),:].flatten()
        contains = False
        for i in s1:
            if i > 0:
                contains = True
                break
        if not contains:
            right_Y -= 1
        else:
            break

    for i in range(Z):
        s1 = locations[:,:,i].flatten()
        contains = False
        for i in s1:
            if i > 0:
                contains = True
                break
        if not contains:
            left_Z += 1
        else:
            break

    for i in range(Z):
        s1 = locations[:,:,-(i+1)].flatten()
        contains = False
        for i in s1:
            if i > 0:
                contains = True
                break
        if not contains:
            right_Z -= 1
        else:
            break

    print(left_X, right_X)
```

```
print(left_Y, right_Y)
print(left_Z, right_Z)


# Apply new bounding box
count = 0
for file in os.listdir(IN_PATH):
    if file.endswith(".nii.gz") and TAG in file:
        new_name = file.split(".")[0] + "_resized.nii.gz"
        nii_img = nib.load(IN_PATH + file)
        nii_data = nii_img.get_fdata()
        new_output = nii_data[left_X:right_X, left_Y:right_Y, left_Z:right_Z]
        new_output = np.array(np.float32(new_output))
        img1 = nib.Nifti1Image(new_output, np.eye(4))
        nib.save(img1, OUT_PATH + new_name)
        count += 1
        print(count)

shape = np.shape(nii_data[left_X:right_X, left_Y:right_Y, left_Z:right_Z])

before = X*Y*Z
after = shape[0]*shape[1]*shape[2]

print(f"Number of voxels after reduction: {before}")
print(f"Number of voxels after reduction: {after}")
print(f"Total reduction: {100 - (after/before)*100}%")
```

List of Acronyms